



# Binary Trees, BSTs & Heaps

Recursive Composition, Traversals, Catalan, LCA, Geodesics,  
BSTs & Heaps

# Session Agenda

---

## **Idea 1: The Recursive Tree Mindset**

Divide & Conquer on Subtrees, Tree Anatomy & Universal Composition Templates

## **Idea 2: Tree Traversals & Linearization**

Breadth-First Queue Slices, Level Transitions & Depth-First Invariants

## **Idea 3: The Catalan Mindset & Counting**

Tree Recurrence, Unique BSTs, Dyck Paths & 10 Combinatorial Variations

## **Idea 4: Lowest Common Ancestor (LCA)**

Postorder Bubbling, Path Decomposition, Distance, Cousins & Turns

## **Idea 5: Root-to-Leaf Paths & Geodesics**

Backtracking Paths, Tree Diameters, Leaf-to-Leaf Sums & Split Rules

## **Idea 6: Binary Search Trees (BST)**

The Sorted Inorder Invariant, Search-Insert-Delete & 10 Problem Types

## **Idea 7: Heaps & Priority Queues**

Array-Based Binary Heaps, Sift Mechanics, Top-K Heuristics & Archetypes

Idea 1

# The Recursive Tree Mindset

Divide & Conquer on Subtrees, Tree Anatomy & Universal Composition Templates

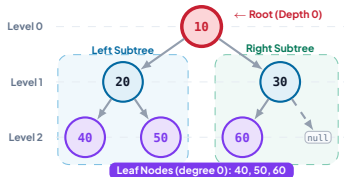
Chapter 04: Trees | Interview Problem Solving Masterclass

# The Tree Data Structure: Anatomy & Hierarchy

## Hierarchical Representation

A **Binary Tree** is a hierarchical structure where each node has **at most two children**:

- **Root**: Topmost node (depth 0, no parent).
- **Leaf**: Degree 0 node (left == null && right == null).
- **Depth / Height**: Distance from root / distance to deepest leaf.
- **Subtree**: Left & right child trees are self-contained.



```
1 class TreeNode {  
2     int val;  
3     TreeNode left, right;  
4     TreeNode(int val) { this.val = val; }  
5 }
```

**Invariant**

A tree of size  $N$  contains exactly  $N - 1$  edges and zero cycles.

# Binary Tree Type 1: Full Binary Tree

## Strict Degree Invariant

A binary tree is **Full** (or *Strict / Proper*) if and only if:  
**Every node has strictly 0 or 2 children.**

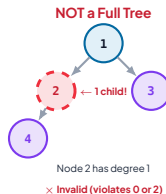
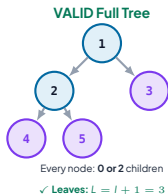
No node is permitted to have exactly 1 child!

## Mathematical Properties

Let  $I$  be internal nodes, and  $L$  be leaf nodes:

$$L = I + 1, \quad N = 2I + 1$$

**Application:** Huffman Coding trees are strictly full binary trees!



**Invariant**

**In a full binary tree, leaf count always exceeds internal nodes by exactly 1 ( $L = I + 1$ ).**

# Binary Tree Type 2: Complete Binary Tree

## Left-Packed Invariant

A binary tree is **Complete** if:

1. All levels except possibly the last are **completely filled**.
2. Last level nodes are packed as far **left as possible** without gaps.

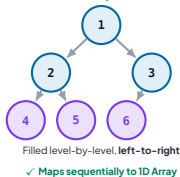
## Flat Array Sequential Storage

Zero missing gaps  $\implies$  stored sequentially in 1D array:

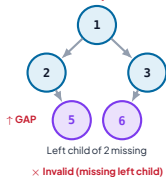
$$\text{Left}(i) = 2i + 1, \quad \text{Right}(i) = 2i + 2$$

**Foundation:** Binary Heaps & Priority Queues!

VALID Complete Tree



NOT a Complete Tree



Invariant

Complete trees achieve  $\mathcal{O}(\log N)$  guaranteed height with zero pointer overhead.

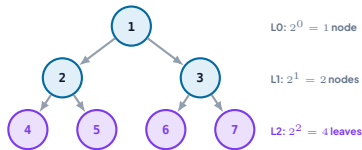
# Binary Tree Type 3: Perfect Binary Tree

## Maximum Packing Invariant

A binary tree is **Perfect** if:

- All **internal nodes** have exactly 2 children.
- All **leaf nodes** are at the **exact same level/depth**.

It represents the fullest possible binary tree for height  $h$ .



Total  $N = 2^{h+1} - 1 = 7$  | Height  $h = 2$  | Leaves  $L = 4$

## Exact Formulas for Height $h$

- Total Nodes:  $N = 2^{h+1} - 1$
- Leaf Nodes:  $L = 2^h = (N + 1)/2$
- Internal Nodes:  $I = 2^h - 1 = L - 1$

**Invariant**

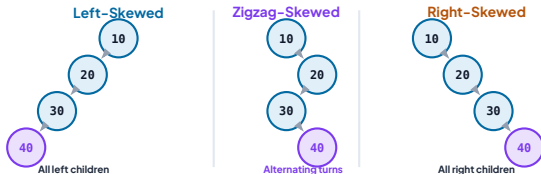
In a perfect tree, the bottom layer alone holds more than half the total nodes ( $L = 2^h$ ).

# Binary Tree Type 4: Skewed Binary Tree

## The Degenerate Chain

A binary tree is **Skewed** (or *Degenerate*) if:  
**Every internal node has strictly 1 child.**

- **Left-Skewed:** Pure left children path.
- **Zigzag-Skewed:** Alternating left/right turns.
- **Right-Skewed:** Pure right children path.



## The Performance Collapse

- Height  $H$  degrades from  $\mathcal{O}(\log N) \rightarrow \mathcal{O}(N)$ .
- Tree structure collapses into a **Linked List**.
- Search time degenerates to  $\mathcal{O}(N)$ .

Invariant

Any single-child chain collapses tree height to  $H = N$ , eliminating logarithmic advantage.



# The Mental Model: Trees Are Naturally Recursive

## The Structural Truth

Look closely at any binary tree:

**A Tree = Root Node + Left Subtree + Right Subtree**

The left child is the root of an independent binary tree.  
The right child is also the root of a full binary tree!

Solve any tree problem by solving the identical problem on its subtrees and combining the answers!

## The 3-Step Recursive Contract

To solve almost any tree problem in 3 lines:

1. **Base Case:** What is the return value for an empty tree (`root == null`)? (e.g., 0, true,  $-\infty$ ).
2. **Subtree Delegation:** Trust `solve(root.left)` and `solve(root.right)` to compute subtree answers.
3. **Node Composition:** Combine `root.val` with left and right results!

Invariant

**Never micromanage tree traversal with loops or queues when a 3-line recursive composition suffices!**

# The Universal Tree Recursion Template

```
1 ReturnType solve(TreeNode root) {  
2     if (root == null) return BASE_VAL;  
3     ReturnType left = solve(root.left);  
4     ReturnType right = solve(root.right);  
5     return combine(root.val, left, right);  
6 }
```

## The 3 Invariant Phases

1. **Base Case:** Return identity element for null ( $0, -\infty, \text{true}$ ).
2. **Subtree Trust:** Recursively solve left and right child subtrees.
3. **Local Combine:** Merge subtree results with root.val in  $\mathcal{O}(1)$ .

**Generality:** Sum, Height, Max, Search, and Symmetry all share this exact template!

Invariant

Mastering this composition blueprint turns complex tree interview problems into 30-second reflex solutions.

# Problem Statement: Sum of All Nodes in Binary Tree

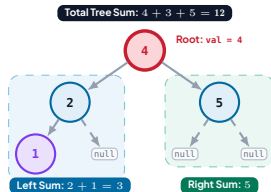
## Problem Statement

Given the root of a binary tree with integer node values, compute the **sum of values of all nodes** present in the binary tree.

### Example Subtree Breakdown:

- Root Value = 4
- Left Subtree Sum =  $2 + 1 = 3$
- Right Subtree Sum = 5

$$\text{Total Sum} = 4 + 3 + 5 = 12$$



## Subtree Recurrence

$$\text{sum}(\text{null}) = 0$$

$$\text{sum}(u) = u.\text{val} + \text{sum}(u.\text{left}) + \text{sum}(u.\text{right})$$

Invariant

Each node receives subtree sums from below, adds its own value, and returns upward.

# Sum of All Nodes: Java Implementation

```
1 int sumTree(TreeNode root) {  
2     if (root == null) return 0;  
3     return root.val  
4         + sumTree(root.left)  
5         + sumTree(root.right);  
6 }
```

## Line-by-Line Mechanics

**Line 2 (Base Case):** If `root == null`, return 0 immediately (empty subtree contributes nothing).

**Line 3 (Composition):** Recursively evaluate left and right subtree sums, add `root.val`, and return total.

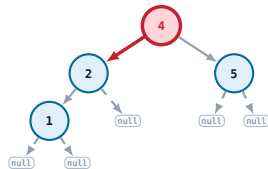
**Post-Order Nature:** Bottom-up aggregation ensures children resolve before parent computes.

Invariant

A 3-line bottom-up recurrence replaces 15+ lines of iterative queue maintenance.

# Execution Trace: Initial Call on Root Node

```
1 int sumTree(TreeNode root) {  
2   if (root == null) return 0;  
3   return root.val  
4     + sumTree(root.left)  
5     + sumTree(root.right);  
6 }
```



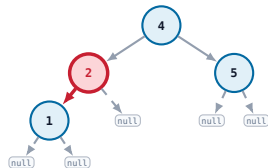
Active Call: `sumTree(root=4)` | `val=4`

## Step Deduction – Root Pauses for Left Subtree

root (4) is not null. Execution pauses at line 3 to evaluate `sumTree(4.left)`, launching call `sumTree(2)`.

# Execution Trace: Traversing Left Subtree

```
1 int sumTree(TreeNode root) {  
2   if (root == null) return 0;  
3   return root.val  
4     + sumTree(root.left)  
5     + sumTree(root.right);  
6 }
```



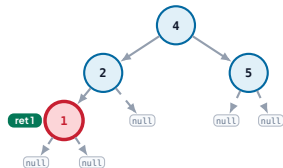
Active Call: `sumTree(root=2)` | `val=2`

## Step Deduction – Left Child Pauses for Subtree

Node 2 is not null. It pushes stack frame and pauses to evaluate its left child: `sumTree(2.left)`, launching call `sumTree(1)`.

# Execution Trace: Base Case & Leaf Return

```
1 int sumTree(TreeNode root) {  
2   if (root == null) return 0;  
3   return root.val  
4     + sumTree(root.left)  
5     + sumTree(root.right);  
6 }
```



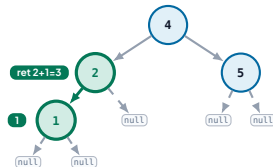
Active Call: `sumTree(1)` | `left=0`, `right=0`

## Step Deduction – Leaf Node Aggregation

Both children of node 1 are null ( $\rightarrow 0$ ).  
Line 3 computes  $1 + 0 + 0 = 1$  and pops frame, returning **1** to node 2.

# Execution Trace: Resolving Left Subtree Sum

```
1 int sumTree(TreeNode root) {  
2   if (root == null) return 0;  
3   return root.val  
4     + sumTree(root.left)  
5     + sumTree(root.right);  
6 }
```



Resume Call: `sumTree(2)` | `left=1`, `right=0`

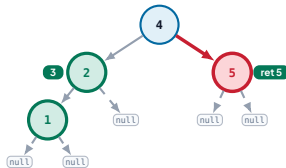
## Step Deduction – Left Subtree Fully Computed

Node 2 receives 1 from left child; right child is `null` ( $\rightarrow 0$ ). Returns  $2 + 1 + 0 = 3$  to root 4.



# Execution Trace: Traversing Right Subtree

```
1 int sumTree(TreeNode root) {  
2     if (root == null) return 0;  
3     return root.val  
4         + sumTree(root.left)  
5         + sumTree(root.right);  
6 }
```



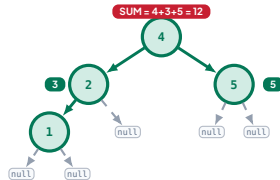
Active Call: `sumTree(5)` | `left=0`, `right=0`

## Step Deduction – Right Subtree Leaf Evaluated

Root launches `sumTree(4.right)` on node 5. With two `null` children ( $0 + 0$ ), node 5 returns  $5 + 0 + 0 = 5$  to root 4.

# Execution Trace: Combining Subtree Sums at Root

```
1 int sumTree(TreeNode root) {  
2   if (root == null) return 0;  
3   return root.val  
4     + sumTree(root.left)  
5     + sumTree(root.right);  
6 }
```



Final Return: `sumTree(4)` | left=3, right=5

## Step Deduction – Final Tree Aggregation

Root 4 combines:  
 $\text{val}(4) + \text{left}(3) + \text{right}(5) = 12$ . The entire tree is summed in a single DFS pass!

# Complexity Analysis: Recursive Subtree Composition

$$\mathcal{O}(N)$$

## Time Complexity

Every node is visited exactly once; each frame performs  $\mathcal{O}(1)$  local work.

$$\mathcal{O}(H)$$

## Space Complexity (Stack Depth)

Height  $H = \mathcal{O}(\log N)$  on balanced trees;  $H = \mathcal{O}(N)$  on skewed trees.

## Zero-Allocation Call Stack State Machine

- **Hardware L1 Cache Efficiency:** Recursion leverages the CPU's native activation records (call stack) with **zero heap allocation overhead**. Iterative solutions require allocating dynamic Queue or Stack objects on the JVM heap.
- **Direct Inductive Equivalence:** The code directly implements mathematical subtree induction  $T(u) = u.val + T(u.left) + T(u.right)$ , eliminating boilerplate pointer polling and null-checks.

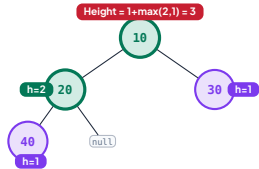
Invariant

Recursive tree DFS achieves optimal  $\mathcal{O}(N)$  runtime with zero dynamic heap allocation overhead.

# Variation 1: Maximum Depth (Height) of Binary Tree

```
1 int maxDepth(TreeNode root) {  
2     if (root == null) return 0;  
3     return 1 + Math.max(maxDepth(root.left),  
4                           maxDepth(root.right));  
5 }
```

**Composition Rule:** The height at node  $u$  is 1 (for current node) plus the **maximum** height between its left and right subtrees!

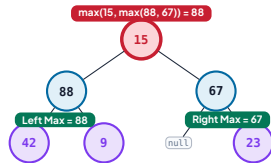


**Template Change:** `1 + Math.max(L, R)`

# Variation 2: Finding Maximum Element in Binary Tree

```
1 int findMax(TreeNode root) {  
2     if (root == null) return Integer.MIN_VALUE;  
3     int leftMax = findMax(root.left);  
4     int rightMax = findMax(root.right);  
5     return Math.max(root.val,  
6         Math.max(leftMax, rightMax));  
7 }
```

**Identity Base Case:** An empty subtree returns `Integer.MIN_VALUE` so it never overrides real node values.

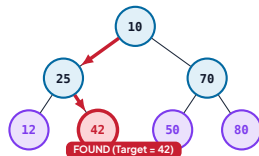


**Template Change:** `null`  $\rightarrow$   $-\infty$ , `Math.max(val, max(L, R))`

## Variation 3: Search / Node Existence in Binary Tree

```
1 boolean exists(TreeNode root, int target) {  
2     if (root == null) return false;  
3     return root.val == target  
4         || exists(root.left, target)  
5         || exists(root.right, target);  
6 }
```

**Short-Circuit Optimization:** The logical OR (||) operator stops traversing the right subtree the moment the target is found in the left subtree!



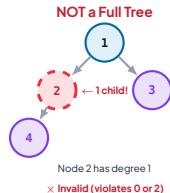
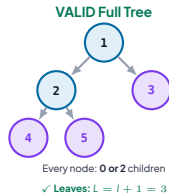
Short-circuit || returns true

**Template Change:** val == target || L || R

# Variation 4: Checking if Binary Tree is Full

```
1 boolean isFull(TreeNode root) {  
2     if (root == null) return true;  
3     if (root.left == null &&  
4         root.right == null) return true;  
5     if (root.left != null &&  
6         root.right != null)  
7         return isFull(root.left) &&  
8             isFull(root.right);  
9     return false; // Exactly 1 child  
10 }
```

**Structural Invariant:** Node has 0 children (line 3) OR 2 valid children (line 4). If exactly one is null, returns false.



**Template Change:** Check 0 or 2 children +  
recurse

# Variation 5: Checking if Binary Tree is Perfect

```
1 boolean isPerfect(TreeNode root, int d, int lvl) {  
2     if (root == null) return true;  
3     if (root.left == null && root.right == null)  
4         return d == lvl + 1;  
5     if (root.left == null || root.right == null)  
6         return false;  
7     return isPerfect(root.left, d, lvl + 1)  
8         && isPerfect(root.right, d, lvl + 1);  
9 }
```

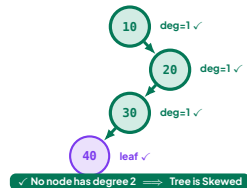
## Leaf Depth Invariant

1. **Depth:** Find depth  $d$  of leftmost leaf.
2. **Check:** For every leaf, verify  $lvl + 1 == d$ .
3. **Internal:** Every internal node must have both children.



# Variation 6: Checking if Binary Tree is Skewed

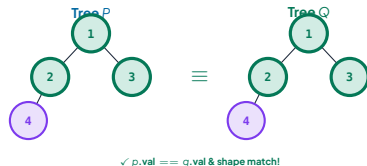
```
1 boolean isSkewed(TreeNode root) {  
2     if (root == null ||  
3         (root.left == null && root.right == null))  
4         return true;  
5     if (root.left != null && root.right != null)  
6         return false; // Degree cannot be 2  
7     return root.left != null  
8         ? isSkewed(root.left)  
9         : isSkewed(root.right);  
10 }
```



**Degree  $\leq 1$  Invariant:** If both children exist, reject with false. Otherwise follow the single branch.

# Variation 7: Checking if Two Trees are Identical

```
1 boolean isSame(TreeNode p, TreeNode q) {  
2     if (p == null && q == null) return true;  
3     if (p == null || q == null) return false;  
4     return (p.val == q.val)  
5         && isSame(p.left, q.left)  
6         && isSame(p.right, q.right);  
7 }
```

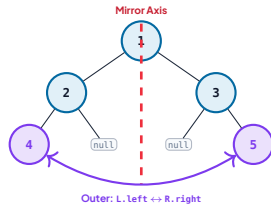


**Dual-Tree Recursion:** Advance synchronously through both trees  $p$  and  $q$ . Both must match in value AND shape!

**Template Change:** Simultaneous dual-root recursion ( $p, q$ )

# Variation 8: Checking if Binary Tree is Foldable

```
1 boolean isFoldable(TreeNode root) {  
2     if (root == null) return true;  
3     return isMirror(root.left, root.right);  
4 }  
5 boolean isMirror(TreeNode n1, TreeNode n2) {  
6     if (n1 == null && n2 == null) return true;  
7     if (n1 == null || n2 == null) return false;  
8     return isMirror(n1.left, n2.right)  
9         && isMirror(n1.right, n2.left);  
10 }
```



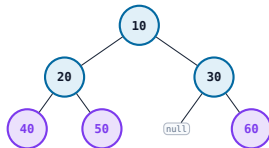
## Step Deduction – Cross-Subtree Reflection

Left subtree's outer child ( $n1.left$ ) reflects right subtree's outer child ( $n2.right$ ).  
Cross-recursion mirrors the axes!

# Variation 9: Counting & Summing Leaf Nodes

```
1 int countLeaves(TreeNode root) {  
2     if (root == null) return 0;  
3     if (root.left == null &&  
4         root.right == null)  
5         return 1; // Leaf identified  
6     return countLeaves(root.left)  
7         + countLeaves(root.right);  
8 }
```

**Leaf Sum Variation:** Change return 1; on line 4 to return root.val; to sum all leaf node values!



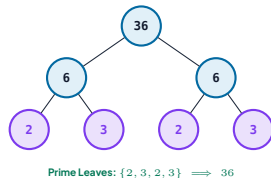
Leaves: [40, 50, 60]  $\Rightarrow$  Count = 3

**Template Change:** Filter on leaf condition

L==null && R==null

# Variation 10: The Factor Tree (Prime Decomposition)

```
1  TreeNode buildFactorTree(int n) {  
2      TreeNode root = new TreeNode(n);  
3      for (int i = 2; i * i <= n; i++) {  
4          if (n % i == 0) {  
5              root.left = buildFactorTree(i);  
6              root.right = buildFactorTree(n / i);  
7              break; // Split into two factors  
8          }  
9      }  
10     return root; // Leaves are Prime Factors!  
11 }
```



**Factor Tree Property:** Internal nodes represent composite products; all leaf nodes are **irreducible prime factors**!

# Summary: The Recursive Tree Template Matrix

## One 4-Line Recursive Spine Solves 10+ Distinct Tree Problems!

| Tree Problem       | Base Value (null)    | Leaf / Node Condition  | Subtree Composition           |
|--------------------|----------------------|------------------------|-------------------------------|
| Sum of Nodes       | return 0;            | None                   | root.val + left + right       |
| Max Depth / Height | return 0;            | None                   | 1 + Math.max(left, right)     |
| Max Element        | return MIN_VAL;      | None                   | Math.max(root.val, max(L, R)) |
| Search Element     | return false;        | root.val == target     | left    right                 |
| Full Tree Check    | return true;         | L==null && R==null     | left && right (degree 2)      |
| Identical Trees    | p==null && q==null   | p.val == q.val         | isSame(p.L, q.L) && ...       |
| Foldable Mirror    | n1==null && n2==null | Shape match            | isMirror(n1.L, n2.R) && ...   |
| Count Leaves       | return 0;            | L==null && R==null → 1 | count(L) + count(R)           |

Invariant

Tree recursion decomposes any global tree query into independent local subtree answers!

## Idea 2

# Tree Traversals & Linearization

Breadth-First Queue Slices, Level Transitions & Depth-First Ordering Invariants

# The Core Challenge: Linearizing a 2D Hierarchy

## The Dimensionality Gap

In arrays and linked lists, visiting elements is trivial: there is only **1 dimension** (index  $0 \rightarrow N - 1$ ).

In trees, branching introduces **2 distinct dimensions**:

- **Breadth**: Horizontal span across sibling nodes.
- **Depth**: Vertical distance down parent-child branches.

How do we visit every single node in a well-defined, reproducible sequential order?

## The Two Fundamental Paradigms

### 1. Breadth-First Search (BFS / Level Order):

- Sweeps horizontally level-by-level.
- Powered by a **FIFO Queue** data structure.

### 2. Depth-First Search (DFS):

- Plunges vertically down to leaves before backtracking.
- Powered by **Recursion / LIFO Stack**.
- 3 variations: **Preorder, Inorder, Postorder**.

Invariant

Linearizing a tree requires choosing between horizontal level sweeps (BFS) or vertical branch dives (DFS).

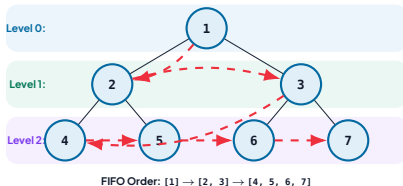


# Breadth-First Search: The Level Order Sweep

## The FIFO Principle

To visit nodes layer by layer:

1. Start by enqueueing the **Root** (Level 0).
2. Dequeue a node from the **front**, process its value, and enqueue its left and right children to the **back**.
3. Because a Queue is **First-In First-Out (FIFO)**, all nodes of Level  $k$  are guaranteed to be processed **before** any node of Level  $k + 1$ !



**FIFO queue preserves the exact**  
**Invariant** horizontal top-to-bottom,  
left-to-right order.

# The Level-Switching Mechanism: Snapshotting Level Size

## Where Does the Level Switch?

In interviews, grouping nodes by level (`List<List<Integer>>`) is the #1 asked BFS pattern.

The Crucial Code Line:

```
int levelSize = queue.size();
```

Why this works:

- At the start of the outer loop, the queue contains **only** nodes of the current level  $k$ .
- Capturing `levelSize` fixes the exact loop count for level  $k$ .
- Children pushed during this loop wait in queue for level  $k + 1$ !

### 1. Snapshot levelSize = 2

2   3

Queue holds Level 1 nodes

Loop runs exactly 2 times

### 2. Next Level Enqueued

4   5   6   7

Children of 2 & 3 pushed

Level boundary guaranteed!

Inner loop consumes snapshot `levelSize`  $\implies$  clean level boundary!

The inner loop consumes  
**Invariant** `snapshot levelSize`, establishing  
an airtight level boundary!

# Level Order Traversal: Java Implementation

```
1 List<List<Integer>> levelOrder(TreeNode root) {
2     List<List<Integer>> res = new ArrayList<>();
3     if (root == null) return res;
4     Queue<TreeNode> q = new LinkedList<>();
5     q.offer(root);
6     while (!q.isEmpty()) {
7         int levelSize = q.size(); // LEVEL SNAPSHOT
8         List<Integer> currLevel = new ArrayList<>();
9         for (int i = 0; i < levelSize; i++) {
10             TreeNode node = q.poll();
11             currLevel.add(node.val);
12             if (node.left != null) q.offer(node.left);
13             if (node.right != null) q.offer(node.right);
14         }
15         res.add(currLevel); // LEVEL k COMPLETE
16     }
17     return res;
18 }
```

## Key Structural Invariants

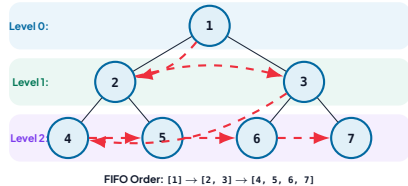
**Line 7:** Snapshot levelSize locks boundary for level k.

**Lines 9–14:** Inner loop pops only current-level nodes, pushing next-level children safely behind.

**Line 15:** Level k closes and commits to final result!

# Level Order Traversal: Level 0 Initialization

```
1 Queue<TreeNode> q = new LinkedList<>();
2 q.offer(root); // Enqueue 1
3 while (!q.isEmpty()) {
4     int levelSize = q.size(); // size = 1
5     List<Integer> currLevel = new ArrayList<>();
6     // ...
7 }
```



Queue: [1] | levelSize = 1 | Level 0

## Step Deduction – Level 0 Snapshot

Root node 1 is enqueued. levelSize = 1 locks the inner loop to run exactly 1 time for Level 0.

# Level Order Traversal: Processing Level 0

```
1 TreeNode node = q.poll(); // Pop 1
2 currLevel.add(node.val); // [1]
3 if (node.left != null)
4     q.offer(node.left); // Push 2
5 if (node.right != null)
6     q.offer(node.right); // Push 3
```

## 1. Snapshot levelSize = 2

|   |   |
|---|---|
| 2 | 3 |
|---|---|

Queue holds Level 1 nodes

Loop runs exactly 2 times

## 2. Next Level Enqueued

|   |   |   |   |
|---|---|---|---|
| 4 | 5 | 6 | 7 |
|---|---|---|---|

Children of 2 & 3 pushed

Level boundary guaranteed!

Inner loop consumes snapshot levelSize  $\Rightarrow$  clean level boundary!

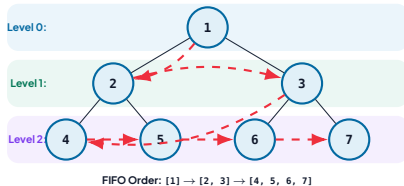
Pop: 1  $\rightarrow$  Queue holds Level 1: [2, 3]

## Step Deduction – Children Enqueued for Next Level

Node 1 is popped and recorded in Level 0 list: [1]. Its children 2 and 3 are pushed to the back of the queue.

# Level Order Traversal: Level 1 Boundary Snapshot

```
1 while (!q.isEmpty()) {  
2     int levelSize = q.size(); // size = 2  
3     List<Integer> currLevel = new ArrayList<>();  
4     for (int i = 0; i < levelSize; i++) {  
5         // Runs exactly 2 times for [2, 3]  
6     }  
7 }
```



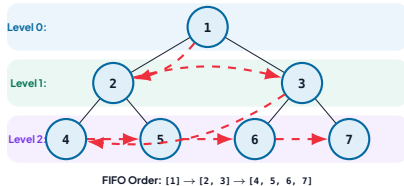
Queue: [2, 3] | levelSize = 2 | Level 1

## Step Deduction – Level 1 Loop Invariant

Outer loop begins second iteration. levelSize = 2  
freezes the execution count for Level 1: nodes 2 and 3.

# Level Order Traversal: Completing Level 1

```
1 // i=0: Pop 2 -> Push 4, 5
2 // i=1: Pop 3 -> Push 6, 7
3 res.add(currLevel); // Level 1: [2, 3]
4 // Queue now holds Level 2: [4, 5, 6, 7]
```



**Result So Far:** [[1], [2, 3]] | **Queue:** [4, 5, 6, 7]

## Step Deduction – Level 1 Committed, Level 2 Ready

Nodes 2 and 3 finish. `currLevel [2, 3]` is committed.  
Queue now cleanly holds all 4 nodes of Level 2!

# Complexity Analysis: Breadth-First Level Order

$$\mathcal{O}(N)$$

## Time Complexity

Every node is enqueued and dequeued exactly once ( $\mathcal{O}(1)$  per node).

$$\mathcal{O}(W)$$

## Space Complexity (Queue Width)

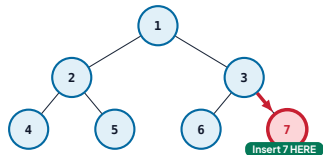
$W$  is the maximum width of the tree. In a full/perfect tree,  
 $W \approx N/2 = \mathcal{O}(N)$ .

**Space Trade-off:** BFS space is proportional to tree **width** ( $\mathcal{O}(W)$ ), whereas DFS space is proportional to tree **height** ( $\mathcal{O}(H)$ ). On balanced trees, DFS uses  $\mathcal{O}(\log N)$  memory while BFS uses  $\mathcal{O}(N)$ !



# Variation 1: Node Insertion in Binary Tree

```
1 void insert(TreeNode root, int val) {
2     if (root == null) return;
3     Queue q = new LinkedList<>();
4     q.offer(root);
5     while (!q.isEmpty()) {
6         TreeNode curr = q.poll();
7         if (curr.left == null) {
8             curr.left = new TreeNode(val);
9             break; // First available left slot
10        } else q.offer(curr.left);
11        if (curr.right == null) {
12            curr.right = new TreeNode(val);
13            break; // First available right slot
14        } else q.offer(curr.right);
15    }
16 }
```



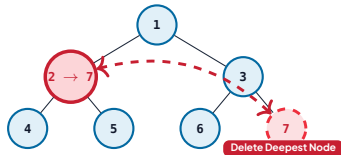
First available null slot in BFS order  $\implies$  Tree stays balanced!

**Why BFS for Insertion?** Inserting at the first available null child preserves the **complete tree structure** and prevents severe tree skewing!

# Variation 2: Node Deletion in Binary Tree

```
1 void deleteNode(TreeNode root, int key) {  
2     // 1. Find target node via BFS  
3     // 2. Find deepest rightmost node (last in BFS)  
4     TreeNode target = find(root, key);  
5     TreeNode deepest = getDeepest(root);  
6     target.val = deepest.val; // Overwrite target  
7     deleteDeepest(root, deepest); // Sever leaf  
8 }
```

**2-Step Deletion Protocol:** Overwrite target with the **deepest rightmost node**, then delete that deepest leaf. Zero tree disruption!

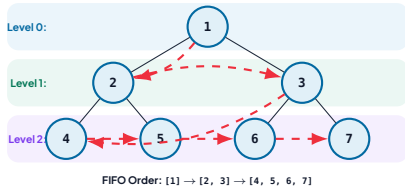


1. Overwrite target with deepest node (7) | 2. Delete deepest node

**Template Change:** BFS search + deepest node replacement

# Variation 3: Maximum Width of Binary Tree

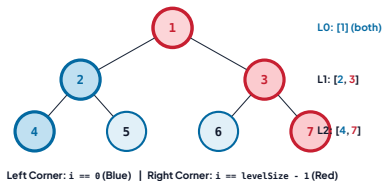
```
1 int maxLevelWidth(TreeNode root) {
2     if (root == null) return 0;
3     Queue<TreeNode> q = new LinkedList<>();
4     q.offer(root);
5     int maxWidth = 0;
6     while (!q.isEmpty()) {
7         int count = q.size(); // Current level width
8         maxWidth = Math.max(maxWidth, count);
9         for (int i = 0; i < count; i++) {
10             TreeNode node = q.poll();
11             if (node.left != null) q.offer(node.left);
12             if (node.right != null) q.offer(node.right);
13         }
14     }
15     return maxWidth;
16 }
```



**Width Metric:** The variable `count = q.size()` directly measures the node count of every layer.

# Variation 4: Corner Nodes at Each Level

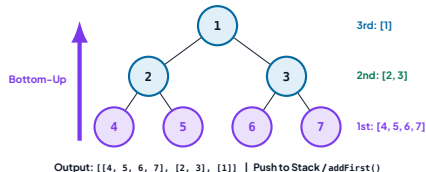
```
1 void printCornerNodes(TreeNode root) {  
2     if (root == null) return;  
3     Queue<TreeNode> q = new LinkedList<>();  
4     q.offer(root);  
5     while (!q.isEmpty()) {  
6         int size = q.size();  
7         for (int i = 0; i < size; i++) {  
8             TreeNode n = q.poll();  
9             if (i == 0 || i == size - 1)  
10                System.out.print(n.val + " ");  
11             if (n.left != null) q.offer(n.left);  
12             if (n.right != null) q.offer(n.right);  
13         }  
14     }  
15 }
```



Corner Predicate:  $i == 0 \mid\mid i == \text{size} - 1$

# Variation 5: Reverse Level Order Traversal

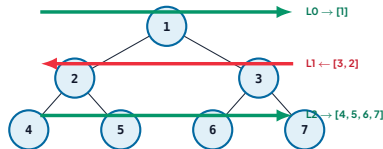
```
1 List<List<Integer>> reverseLevelOrder(TreeNode root) {
2     LinkedList<List<Integer>> res = new LinkedList<>();
3     if (root == null) return res;
4     Queue<TreeNode> q = new LinkedList<>();
5     q.offer(root);
6     while (!q.isEmpty()) {
7         int size = q.size();
8         List<Integer> level = new ArrayList<>();
9         for (int i = 0; i < size; i++) {
10             TreeNode n = q.poll();
11             level.add(n.val);
12             if (n.left != null) q.offer(n.left);
13             if (n.right != null) q.offer(n.right);
14         }
15         res.addFirst(level); // INSERT AT HEAD
16     }
17     return res;
18 }
```



Template Change: `res.addFirst(level)` / Stack reversal

# Variation 6: Zigzag / Spiral Order Traversal

```
1 List<List<Integer>> zigzagLevelOrder(TreeNode root) {
2     List<List<Integer>> res = new ArrayList<>();
3     if (root == null) return res;
4     Queue<TreeNode> q = new LinkedList<>();
5     q.offer(root);
6     boolean leftToRight = true;
7     while (!q.isEmpty()) {
8         int size = q.size();
9         LinkedList<Integer> level = new LinkedList<>();
10        for (int i = 0; i < size; i++) {
11            TreeNode n = q.poll();
12            if (leftToRight) level.addLast(n.val);
13            else level.addFirst(n.val); // Reverse
14            if (n.left != null) q.offer(n.left);
15            if (n.right != null) q.offer(n.right);
16        }
17        res.add(level);
18        leftToRight = !leftToRight; // TOGGLE DIRECTION
19    }
20    return res;
21 }
```



Output:  $[[1], [3, 2], [4, 5, 6, 7]]$  | Toggle leftToRight flag!

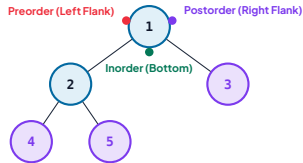
**Template Change:** Toggle leftToRight flag + Deque

# Depth-First Traversals: The Euler Tour Model

## The 3-Point Boundary Walk

Imagine walking counter-clockwise along the perimeter of the tree. Every node is encountered **three times**:

1. **Preorder** ( $N \rightarrow L \rightarrow R$ ): Visited on **first arrival** (left flank) before entering subtrees.
2. **Inorder** ( $L \rightarrow N \rightarrow R$ ): Visited **between** left and right subtree recursions (bottom flank).
3. **Postorder** ( $L \rightarrow R \rightarrow N$ ): Visited on **final return** (right flank) after both subtrees finish.



Every node is passed 3 times: on arrival (Pre), between subtrees (In), on return (Post)!

**DFS traversals differ ONLY in Invariant when the node's value is processed during the tour.**

# The 3 DFS Recursive Blueprints in Java

## Preorder ( $N \rightarrow L \rightarrow R$ )

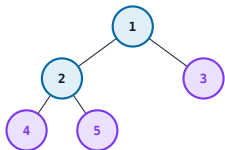
```
void pre(TreeNode r) {  
    if (r == null) return;  
    visit(r.val);  
    pre(r.left);  
    pre(r.right);  
}
```

## Inorder ( $L \rightarrow N \rightarrow R$ )

```
void in(TreeNode r) {  
    if (r == null) return;  
    in(r.left);  
    visit(r.val);  
    in(r.right);  
}
```

## Postorder ( $L \rightarrow R \rightarrow N$ )

```
void post(TreeNode r) {  
    if (r == null) return;  
    post(r.left);  
    post(r.right);  
    visit(r.val);  
}
```



Preorder(N-L-R): [1, 2, 4, 5, 3]

Inorder(L-N-R): [4, 2, 5, 1, 3]

Postorder(L-R-N): [4, 5, 2, 3, 1]



# Variation 7: Inorder Traversal & BST Validation

```
1 Integer prev = null;
2 boolean isValidBST(TreeNode root) {
3     if (root == null) return true;
4     if (!isValidBST(root.left)) return false;
5     if (prev != null && root.val <= prev)
6         return false; // Violates ascending order!
7     prev = root.val;
8     return isValidBST(root.right);
9 }
```

## Inorder Sorted Guarantee

On a valid Binary Search Tree (BST), Inorder traversal visits elements in **strictly ascending order**!

We simply verify that every visited node is strictly greater than prev in  $\mathcal{O}(N)$  time and  $\mathcal{O}(H)$  memory.

# Variation 8: Postorder Traversal & Bottom-Up Pruning

```
1 TreeNode pruneZeros(TreeNode root) {  
2     if (root == null) return null;  
3     root.left = pruneZeros(root.left);  
4     root.right = pruneZeros(root.right);  
5     if (root.val == 0 &&  
6         root.left == null && root.right == null)  
7         return null; // Prune leaf 0  
8     return root;  
9 }
```

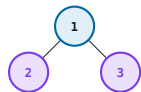
## Bottom-Up Pruning Invariant

**Why Postorder?** You must evaluate and prune the children **first**.

A parent node might become a leaf only AFTER its zero-children are pruned. Postorder guarantees children are clean before parent inspection!

# Variation 9: Preorder Serialization & Reconstruction

```
1 String serialize(TreeNode root) {  
2     if (root == null) return "#,";  
3     return root.val + ","  
4         + serialize(root.left)  
5         + serialize(root.right);  
6 }
```



"1,2,#,#,3,#,#"

# denotes null child

✓ Unambiguously reconstructs tree

Root is always index 0; left subtree is reconstructed recursively before right!

**Reconstruction:** Root is always the first token. Rebuild root, then recursively rebuild left and right subtrees!

**Encoding:** Root first  $\implies$  Unambiguous DFS  
decode

# Variation 10: Iterative DFS Using an Explicit Stack

```
1 List<Integer> preorderIterative(TreeNode root) {  
2     List<Integer> res = new ArrayList<>();  
3     if (root == null) return res;  
4     Stack<TreeNode> st = new Stack<>();  
5     st.push(root);  
6     while (!st.isEmpty()) {  
7         TreeNode curr = st.pop();  
8         res.add(curr.val);  
9         if (curr.right != null) st.push(curr.right);  
10        if (curr.left != null) st.push(curr.left);  
11    }  
12    return res;  
13 }
```

## LIFO Stack Reversal

Because a Stack is LIFO:

Push **Right child first**, then **Left child second**!

This ensures the Left child is popped and processed first, matching standard Preorder ( $N \rightarrow L \rightarrow R$ ).

# Summary: The Tree Traversal Decision Matrix

## Matching Tree Traversals to Algorithmic Invariants

| Traversal         | Data Structure    | Order                           | Level Transition Point | Primary Use Case                       |
|-------------------|-------------------|---------------------------------|------------------------|----------------------------------------|
| Level Order (BFS) | Queue<TreeNode>   | Horizontal                      | levelSize = q.size()   | Shortest path, balanced insert, views  |
| Preorder (DFS)    | Recursion / Stack | $N \rightarrow L \rightarrow R$ | None (Depth-first)     | Tree serialization, cloning, prefix    |
| Inorder (DFS)     | Recursion / Stack | $L \rightarrow N \rightarrow R$ | None (Depth-first)     | BST sorted array, BST validation       |
| Postorder (DFS)   | Recursion / Stack | $L \rightarrow R \rightarrow N$ | None (Depth-first)     | Bottom-up delete, evaluate expressions |
| Zigzag / Spiral   | Queue + Deque     | Alternating                     | Toggle direction flag  | Level rendering, spiral scan           |

Invariant

Choosing the right traversal reduces 50 lines of complex tree logic into a clean 5-line invariant!

## Idea 3

# The Catalan Mindset: Tree Structures & Combinatorics

Counting Unlabeled Trees, Recurrence Derivation, BST Generation & Classic Catalan Isomorphisms

# The Foundational Problem: Counting Binary Trees

## The Core Question

Given  $N$  unlabeled nodes, how many structurally unique binary tree topologies can be constructed?

- A tree topology is defined strictly by its pointer hierarchy (left, right).
- Left and right child orientations are **distinct** (a node with only a left child is different from one with only a right child).

## Why This Matters in Interviews

- **Search Space Size:** Determines the exact time complexity of tree-generating algorithms.
- **Dynamic Programming:** Recursive partition formula translates directly into  $\mathcal{O}(N^2)$  DP.
- **Universal Isomorphism:** The exact same number answers 10+ classic interview questions!

Invariant

The answer to “How many binary trees with  $N$  nodes?” is the  $N$ -th Catalan Number:  $C_N$ .

# Empirical Exploration: Small Cases ( $N = 0, 1, 2$ )

$N = 0$  **Nodes**

**Empty Tree** (null)

There is exactly **1** way to form an empty tree.

$$C_0 = 1$$

$N = 1$  **Node**

**Single Root Node**

Root with two null children.  
Exactly **1** tree.

$$C_1 = 1$$

$N = 2$  **Nodes**

**2 Topologies:**

- Root + Left child
- Root + Right child

$$C_2 = 2$$

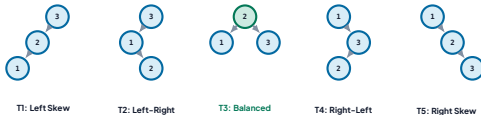
Initial Catalan Sequence:  $C_0 = 1, C_1 = 1, C_2 = 2 \dots$

**Invariant**

**Because left and right pointer orientations are distinct,  $N = 2$  yields 2 distinct topologies.**



# Empirical Exploration: All 5 Topologies for $N = 3$



**Topologies for  $N = 3$ :** Left-skew ( $3 \rightarrow 2 \rightarrow 1$ ), Left-Right zigzag ( $3 \rightarrow 1 \rightarrow 2$ ), Balanced ( $2 \rightarrow 1, 3$ ), Right-Left zigzag ( $1 \rightarrow 3 \rightarrow 2$ ), and Right-skew ( $1 \rightarrow 2 \rightarrow 3$ ).

**Invariant**

**For  $N = 3$ , exactly  $C_3 = 5$  unique binary tree shapes exist.**

# Tree Counting: Unlabeled vs. Labeled vs. BSTs

## 1. Unlabeled Binary Trees

Nodes have no values; only topology matters.

Total:

$$C_N = \frac{1}{N+1} \binom{2N}{N}$$

## 2. Binary Search Trees

$N$  distinct keys ( $1 \dots N$ ). Inorder traversal is **fixed** to sorted order.

Total:

$$\text{BSTs} = C_N$$

## 3. Labeled Binary Trees

$N$  distinct labels assigned arbitrarily to nodes.

Total:

$$C_N \times N!$$

**Invariant**

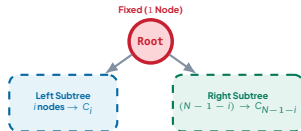
Because a BST's keys are locked in sorted inorder, every unique tree topology corresponds to exactly ONE valid BST.

# The Root-Selection Principle: Deriving the Recurrence

## The Partition Mental Model

To construct a binary tree with  $N$  nodes:

1. Select 1 node as the **Root**.
2. Leaves  $N - 1$  nodes to distribute:
  - Left subtree gets  $i$  nodes ( $0 \leq i \leq N - 1$ ).
  - Right subtree gets  $(N - 1 - i)$  nodes.



Fixed  $i \implies C_i \times C_{N-1-i}$  combinations

$$C_N = \sum_{i=0}^{N-1} C_i \cdot C_{N-1-i}$$

**Invariant**

By the multiplication rule of counting, a partition with  $i$  left nodes produces  $C_i \times C_{N-1-i}$  trees.

# The Universal Catalan Recurrence Relation

Summing Over All Left Subtree Sizes  $i \in [0, N - 1]$

$$C_N = \sum_{i=0}^{N-1} C_i \cdot C_{N-1-i} \quad \text{for } N \geq 1, \quad C_0 = 1$$

Step-by-Step Expansion for Small  $N$ :

- $C_1 = C_0 \cdot C_0 = 1 \cdot 1 = 1$
- $C_2 = C_0 \cdot C_1 + C_1 \cdot C_0 = (1 \cdot 1) + (1 \cdot 1) = 2$
- $C_3 = C_0 \cdot C_2 + C_1 \cdot C_1 + C_2 \cdot C_0 = (1 \cdot 2) + (1 \cdot 1) + (2 \cdot 1) = 2 + 1 + 2 = 5$
- $C_4 = C_0 \cdot C_3 + C_1 \cdot C_2 + C_2 \cdot C_1 + C_3 \cdot C_0 = (1 \cdot 5) + (1 \cdot 2) + (2 \cdot 1) + (5 \cdot 1) = 5 + 2 + 2 + 5 = 14$

Invariant

The convolution sum  $\sum C_i C_{n-1-i}$  is the mathematical signature of the Catalan number family.

# Closed-Form Formula & Asymptotic Complexity

## Closed-Form Binomial Formula

$$C_N = \frac{1}{N+1} \binom{2N}{N} = \frac{(2N)!}{(N+1)!N!}$$

Consecutive Ratio Formula:

$$C_N = \frac{2(2N-1)}{N+1} C_{N-1}$$

Enables computation in  $\mathcal{O}(N)$  time and  $\mathcal{O}(1)$  space!

## Catalan Sequence & Growth

First 8 Catalan Numbers:

1, 1, 2, 5, 14, 42, 132, 429, 1430 ...

Asymptotic Growth (Stirling's):

$$C_N \sim \frac{4^N}{N^{3/2}\sqrt{\pi}}$$

Grows exponentially as  $\mathcal{O}(4^N/N^{1.5})$ .

**Invariant**

Because  $C_N$  grows as  $\mathcal{O}(4^N/N^{1.5})$ , brute-force tree generation becomes intractable for  $N \geq 15$ .

# Java Implementation 1: Dynamic Programming $\mathcal{O}(N^2)$

```
1 int numTreesDP(int n) {  
2     int[] dp = new int[n + 1];  
3     dp[0] = 1;  
4     dp[1] = 1;  
5     for (int nodes = 2; nodes <= n; nodes++) {  
6         for (int i = 0; i < nodes; i++) {  
7             dp[nodes] += dp[i] * dp[nodes - 1 - i];  
8         }  
9     }  
10    return dp[n];  
11 }
```

## DP Invariant Analysis

- **Outer Loop:** Builds table bottom-up from  $2 \rightarrow n$ .
- **Inner Loop:** Evaluates convolution sum  $\sum dp[i] \cdot dp[nodes - 1 - i]$ .
- **Time:**  $\mathcal{O}(N^2)$  additions/multiplications.
- **Space:**  $\mathcal{O}(N)$  auxiliary array.

Invariant

This exact DP table directly solves LeetCode 96 (Unique Binary Search Trees).

# Java Implementation 2: Direct Formula $\mathcal{O}(N)$ in $\mathcal{O}(1)$ Space

```
1 long numTreesLinear(int n) {  
2     long catalan = 1;  
3     for (int i = 0; i < n; i++) {  
4         catalan = catalan * 2 * (2 * i + 1) / (i + 2);  
5     }  
6     return catalan;  
7 }
```

**Formula:**  $C_{i+1} = C_i \times \frac{2(2i+1)}{i+2}$

**Invariant**

Using the ratio recurrence computes the  $N$ -th Catalan number in  $\mathcal{O}(N)$  time with  $\mathcal{O}(1)$  space.

## Overflow Prevention

**Why long?**  $C_N$  grows rapidly ( $C_{19} = 1,767,263,190$  approaches 32-bit integer limit).

**Multiplication before division:** Multiply by  $2(2i+1)$  before dividing by  $(i+2)$  to avoid integer truncation.

# Variation 1: Unique Binary Search Trees (LeetCode 96)

```
1 // Count unique BSTs with keys 1 to n
2 int numTrees(int n) {
3     int[] G = new int[n + 1];
4     G[0] = 1; G[1] = 1;
5     for (int len = 2; len <= n; len++) {
6         for (int root = 1; root <= len; root++) {
7             G[len] += G[root - 1] * G[len - root];
8         }
9     }
10    return G[n];
11 }
```

## BST Partition Invariant

If key root is chosen as root:

- Left subtree must contain keys  $[1 \dots \text{root} - 1]$  ( $\text{root} - 1$  keys).
- Right subtree must contain keys  $[\text{root} + 1 \dots \text{len}]$  ( $\text{len} - \text{root}$  keys).
- Number of trees =  $G(\text{root} - 1) \times G(\text{len} - \text{root})$ .

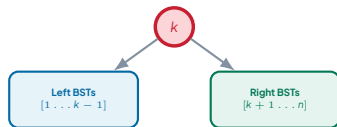
**Invariant**

Because BST property forces strict ordering, keys determine subtree split points identically to Catalan partitions.



## Variation 2: Generate All Unique BSTs (LeetCode 95)

```
1 List<TreeNode> build(int start, int end) {
2     List<TreeNode> list = new ArrayList<>();
3     if (start > end) { list.add(null); return list; }
4     for (int i = start; i <= end; i++) {
5         List<TreeNode> lefts = build(start, i - 1);
6         List<TreeNode> rights = build(i + 1, end);
7         for (TreeNode l : lefts) {
8             for (TreeNode r : rights) {
9                 TreeNode root = new TreeNode(i);
10                root.left = l; root.right = r;
11                list.add(root);
12            }
13        }
14    }
15    return list;
16 }
```



For each  $l \in \text{LeftTrees}$  and each  $r \in \text{RightTrees}$ :

`TreeNode curr = new TreeNode(k); curr.left = l; curr.right = r;`

**Cartesian Product:** Combines every left tree with every right tree

# Variation 3: Balanced Parentheses Expressions ( $N$ Pairs)

## Balanced Parentheses Invariant

Given  $N$  pairs of '(' and ')', a sequence of length  $2N$  is valid if and only if:

1. Total '(' equals total ')' ( $= N$ ).
2. In every prefix,  $\text{count('(')} \geq \text{count('')}$ .

For  $N = 3$  ( $C_3 = 5$  valid combinations):

- $((()))$ ,  $(())()$ ,  $(())()$
- $()((()))$ ,  $()()()$

## Tree $\leftrightarrow$ Parentheses Bijection

Split string around matching closing bracket of first '(' :

(Inside) Outside

- If Inside has  $i$  pairs  $\implies C_i$  ways.
- Then Outside has  $(N - 1 - i)$  pairs  $\implies C_{N-1-i}$  ways.
- Total ways:  $\sum_{i=0}^{N-1} C_i C_{N-1-i} = C_N$ .

Invariant

Splitting a balanced string at its outermost bracket matches the exact left/right subtree root partition!

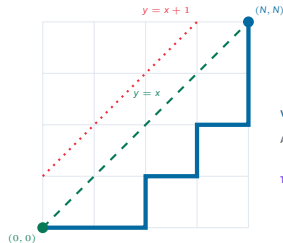
# Variation 4: Monotonic Grid Paths (Dyck Paths)

## André's Reflection Principle

Move from  $(0, 0)$  to  $(N, N)$  using  $N$  Right ( $R$ ) and  $N$  Up ( $U$ ) steps without crossing the diagonal  $y = x$ :

- **Total Grid Paths:**  $\binom{2N}{N}$
- **Invalid Paths:** Hit line  $y = x + 1$ . Reflecting the remaining path across  $y = x + 1$  maps to  $(N - 1, N + 1)$ .
- **Total Invalid Paths:**  $\binom{2N}{N-1}$
- **Valid Dyck Paths:**

$$\binom{2N}{N} - \binom{2N}{N-1} = \frac{1}{N+1} \binom{2N}{N} = C_N$$



**Valid Dyck Path:**

Always  $R \geq U$  (Never crosses  $y = x$ )

**Total Paths:**  $\binom{2n}{n} - \binom{2n}{n-1} = C_n$

**Invariant**

Every valid Dyck path maps 1-to-1 with a balanced parentheses sequence ( $R \rightarrow '(', U \rightarrow ')'$ ).

# Variation 5: Matrix Chain Parenthesizations

## Associative Groupings

Given  $N + 1$  matrices  $M_1 \times M_2 \times \dots \times M_{N+1}$ , how many ways can we fully parenthesize the product?

For  $N = 3$  (4 matrices,  $C_3 = 5$  ways):

1.  $((M_1 M_2) M_3) M_4$
2.  $(M_1 (M_2 M_3)) M_4$
3.  $(M_1 M_2) (M_3 M_4)$
4.  $M_1 ((M_2 M_3) M_4)$
5.  $M_1 (M_2 (M_3 M_4))$

## The Expression Tree Isomorphism

Every full parenthesization corresponds to a unique **full binary expression tree**:

- Leaf nodes represent matrices  $M_1 \dots M_{N+1}$  ( $N + 1$  leaves).
- Internal nodes represent matrix multiplication operations ( $N$  internal nodes).
- Number of full binary trees with  $N + 1$  leaves is **strictly**  $C_N$ .

**Invariant**

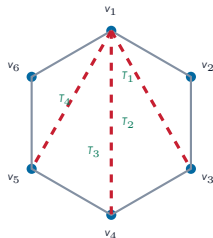
Matrix chain multiplication parenthesization is isomorphic to constructing full binary expression trees.

# Variation 6: Convex Polygon Triangulation

## Euler's Polygon Problem

**Problem:** How many ways can a convex polygon with  $(N + 2)$  vertices be divided into  $N$  non-overlapping triangles using  $(N - 1)$  non-crossing diagonals?

- For a triangle ( $N + 2 = 3 \implies N = 1$ ):  $C_1 = 1$  way.
- For a quadrilateral ( $N + 2 = 4 \implies N = 2$ ):  $C_2 = 2$  ways.
- For a pentagon ( $N + 2 = 5 \implies N = 3$ ):  $C_3 = 5$  ways.
- For a hexagon ( $N + 2 = 6 \implies N = 4$ ):  $C_4 = 14$  ways.



Convex  $(n + 2)$ -gon  $\rightarrow n$  non-overlapping triangles via  $n - 1$  chords  $\implies C_n$  ways

**Invariant**

Fixing a base edge  $(v_1, v_{N+2})$  and picking a third vertex  $v_k$  splits the polygon into two sub-polygons  $(C_{k-2} \times C_{N+1-k})$ .

# Variation 7: Full Binary Trees with $N$ Internal Nodes

## Full Tree Leaf Invariant

In a Full Binary Tree (every node has 0 or 2 children):

$$\text{Leaves } L = I + 1$$

where  $I$  is the count of internal nodes.

- If  $I = 3$ , then  $L = 4$ , and total nodes  $= 2I + 1 = 7$ .
- Total unique full binary tree structures with  $N$  internal nodes (or  $N + 1$  leaves) is **exactly**  $C_N$ .

## Why Full Trees Map to $C_N$

If root is an internal node:

- Left subtree has  $i$  internal nodes ( $0 \leq i \leq N - 1$ ).
- Right subtree has  $(N - 1 - i)$  internal nodes.
- Recurrence:  
$$T(N) = \sum_{i=0}^{N-1} T(i)T(N - 1 - i) = C_N.$$

Invariant

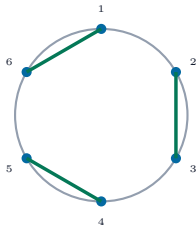
Unlabeled general binary trees with  $N$  nodes are in 1-to-1 bijection with full binary trees with  $N$  internal nodes.

# Variation 8: Non-Intersecting Chords on a Circle

## Chord Connection Rule

Given  $2N$  distinct points on the circumference of a circle, connect them in pairs using  $N$  chords such that **no two chords intersect**.

- Point 1 must connect to an **even point**  $2k$  (leaving an even number of points on both sides of the chord).
- Left side has  $2(k-1)$  points  $\implies C_{k-1}$  ways.
- Right side has  $2(N-k)$  points  $\implies C_{N-k}$  ways.



Connect  $2n$  points with  $n$  non-intersecting chords  $\implies C_n = C_3 = 5$  ways

**Invariant**

Because chords cannot cross, fixing one chord partitions the circle into two independent sub-problems:  $C_N$ .

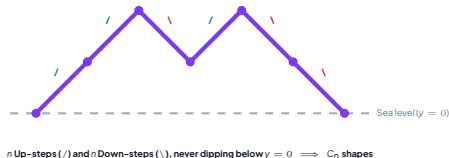
# Variation 9: Mountain Ranges & Skyline Walks

## The Skyline Condition

Draw a mountain range with  $N$  up-strokes (/) and  $N$  down-strokes (\) of length 1 such that:

1. The path starts and ends at sea level ( $y = 0$ ).
2. The path **never dips below sea level** ( $y \geq 0$  everywhere).

Total unique mountain ranges with  $N$  peaks is **strictly**  $C_N$ .



Invariant

An up-stroke is an open bracket ' ( ' , a down-stroke is a closed bracket ' ) ' , and sea level is zero balance.



# Variation 10: Stack Permutations (Valid Pop Sequences)

```
1 // Check if target is a valid stack pop sequence
2 boolean validateStackSeq(int[] pushed, int[] popped) {
3     Stack<Integer> st = new Stack<>();
4     int j = 0;
5     for (int x : pushed) {
6         st.push(x);
7         while (!st.isEmpty() && st.peek() == popped[j]) {
8             st.pop();
9             j++;
10        }
11    }
12    return st.isEmpty();
13 }
```

## LIFO Push/Pop Invariant

Given numbers  $1, 2, \dots, N$  pushed in order:

- How many distinct output permutations can be generated using a single stack?
- A sequence of  $N$  push and  $N$  pop operations is valid if stack never underflows.
- Total valid pop sequences:  $C_N$ .

Invariant

Stack operations (push  $\rightarrow$  '(', pop  $\rightarrow$  ')') form a direct bijection with Catalan Dyck words.

# Summary: Master Catalan Isomorphism Matrix

## 10 Classic Problems Solved by the $N$ -th Catalan Number ( $C_N$ )

| Problem Domain           | Parameter Meaning ( $N$ )                 | Catalan Value   | Universal Bijection Mechanism                                      |
|--------------------------|-------------------------------------------|-----------------|--------------------------------------------------------------------|
| 1. Binary Trees          | $N$ unlabeled nodes                       | $C_N$           | Root chosen $\implies$ Left ( $i$ ) $\times$ Right ( $N - 1 - i$ ) |
| 2. Binary Search Trees   | $N$ sorted keys ( $1 \dots N$ )           | $C_N$           | Fixed inorder ordering $\implies$ 1-to-1 with tree topology        |
| 3. Generate Unique BSTs  | $N$ sorted keys                           | $C_N$ trees     | Recursive Cartesian product of subtree lists                       |
| 4. Balanced Parentheses  | $N$ pairs of '(' and ')'                  | $C_N$ strings   | Outermost bracket split: (Inside) Outside                          |
| 5. Dyck Grid Paths       | $N \times N$ grid, stay below $y = x$     | $C_N$ paths     | Reflection principle: $\binom{2N}{N} - \binom{2N}{N-1}$            |
| 6. Matrix Multiplication | $N + 1$ matrices parenthesized            | $C_N$ ways      | Full binary expression tree with $N + 1$ leaves                    |
| 7. Polygon Triangulation | Convex ( $N + 2$ )-gon into $N$ triangles | $C_N$ ways      | Fix base edge ( $v_1, v_{N+2}$ ) and choose 3rd vertex $v_k$       |
| 8. Full Binary Trees     | $N$ internal nodes ( $N + 1$ leaves)      | $C_N$ trees     | Strict $L = I + 1$ leaf-to-internal node correspondence            |
| 9. Non-Crossing Chords   | $2N$ points on circle connected           | $C_N$ matchings | Point 1 connects to even point $2k$ partitioning circle            |
| 10. Stack Permutations   | $N$ elements pushed sequentially          | $C_N$ sequences | $N$ push and $N$ pop valid operations                              |

Invariant

Whenever a problem asks for non-crossing configurations, binary splits, or prefix-balanced operations, the answer is  $C_N$ !

## Idea 4

# Lowest Common Ancestor (LCA) & Path Invariants

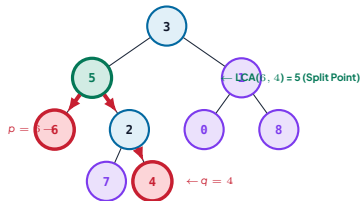
Bottom-Up Postorder Bubbling, Path Decomposition & Structural Geometry on Trees

# Lowest Common Ancestor: Definition & The Split Point

## What is the LCA?

The **Lowest Common Ancestor** of two nodes  $p$  and  $q$  in a binary tree  $T$  is the **deepest node** that has both  $p$  and  $q$  as descendants:

- A node can be a descendant of itself (if  $p$  is an ancestor of  $q$ ,  $LCA(p, q) = p$ ).
- The LCA is the unique **split point** where the paths from the root to  $p$  and  $q$  diverge!



**Invariant**

**LCA is the critical vertex connecting  $p$  and  $q$ : the simple path between them is  $p \rightarrow LCA \rightarrow q$ .**

# The Universal 4-Line LCA Blueprint in Java

```
1 TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {  
2     if (root == null || root == p || root == q) return root;  
3     TreeNode left = lowestCommonAncestor(root.left, p, q);  
4     TreeNode right = lowestCommonAncestor(root.right, p, q);  
5     if (left != null && right != null) return root; // SPLIT POINT  
6     return left != null ? left : right;           // BUBBLE UP  
7 }
```

## Postorder Bubbling Logic

1. **Base Case:** If root is null, p, or q, return root.
2. **Recurse:** Search left and right subtrees.
3. **Split Found:** If both left and right are non-null, p and q lie in separate subtrees  $\implies$  root is LCA!
4. **Single Sided:** If only one subtree returns non-null, pass that result upwards.

**Invariant**

Postorder traversal bubbles discovered targets upwards; the first node receiving non-null from BOTH sides is the LCA.

# Execution Trace: Left Subtree Descent

```
1 TreeNode LCA(TreeNode root, TreeNode p, TreeNode q) {  
2     if (root == null || root == p || root == q) return root;  
3     TreeNode left = LCA(root.left, p, q);  
4     TreeNode right = LCA(root.right, p, q);  
5     if (left != null && right != null) return root;  
6     return left != null ? left : right;  
7 }
```

Call Stack: LCA(3) → LCA(5) → LCA(6)

Visiting Node 6: Matches target  $p = 6$ ! Returns node 6 to parent (5).

## Step Deduction – Target Node Reached

Node 6 bubbles upwards without recursing deeper into its subtrees.

# Execution Trace: Right Subtree Search

```
1 TreeNode LCA(TreeNode root, TreeNode p, TreeNode q) {  
2     if (root == null || root == p || root == q) return root;  
3     TreeNode left = LCA(root.left, p, q);  
4     TreeNode right = LCA(root.right, p, q);  
5     if (left != null && right != null) return root;  
6     return left != null ? left : right;  
7 }
```

Call Stack: LCA(5) → LCA(2) → LCA(4)

Visiting Node 4: Matches target  $q = 4$ ! Returns node 4 to node 2, which returns 4 to node 5.

## Step Deduction – Right Target Located

Node 4 bubbles to node 5 as its right subtree result.

# Execution Trace: Split Point Convergence at Node 5

```
1 TreeNode LCA(TreeNode root, TreeNode p, TreeNode q) {  
2     if (root == null || root == p || root == q) return root;  
3     TreeNode left = LCA(root.left, p, q);  
4     TreeNode right = LCA(root.right, p, q);  
5     if (left != null && right != null) return root;  
6     return left != null ? left : right;  
7 }
```

Node 5 receives: left = 6, right = 4  $\implies$  BOTH NON-NULL!

**Split Invariant:** Node 5 has  $p$  in left subtree and  $q$  in right subtree. Returns Node 5!

## Step Deduction – LCA Convergence

Node 5 is locked as the lowest common ancestor split point.



# Execution Trace: Final Return to Root

```
1 TreeNode LCA(TreeNode root, TreeNode p, TreeNode q) {  
2     if (root == null || root == p || root == q) return root;  
3     TreeNode left = LCA(root.left, p, q);  
4     TreeNode right = LCA(root.right, p, q);  
5     if (left != null && right != null) return root;  
6     return left != null ? left : right;  
7 }
```

Root (3) receives: left = Node 5, right = null  $\implies$  Returns Node 5

**Result Bubble:** Because right subtree returned null, Node 5 propagates all the way to caller.

## Step Deduction – Search Complete

Returns Node 5 in  $\mathcal{O}(N)$  time and  $\mathcal{O}(H)$  stack space.

# Complexity Analysis: Lowest Common Ancestor

## Time Complexity: $\mathcal{O}(N)$

- In the worst case, every node in the binary tree is visited at most once during the DFS traversal.
- If  $p$  and  $q$  are in the far right subtree, left subtree is fully scanned and returns null.
- Total visits:  $\leq N \implies \mathcal{O}(N)$  time.

## Space Complexity: $\mathcal{O}(H)$

- Space is consumed exclusively by the recursion call stack.
- **Balanced Tree:**  $H = \mathcal{O}(\log N)$  stack frames.
- **Skewed Tree:**  $H = \mathcal{O}(N)$  stack frames.
- No auxiliary heap memory allocated ( $\mathcal{O}(1)$  extra space).

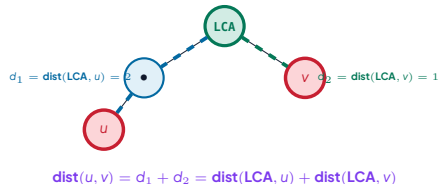
Invariant

LCA runs in optimal  $\mathcal{O}(N)$  time with  $\mathcal{O}(H)$  stack space, requiring zero parent pointers.

# Variation 1: Distance Between Two Nodes

```
1 int findDistance(TreeNode root, int u, int v) {  
2     TreeNode lca = LCA(root, u, v);  
3     int d1 = getDepth(lca, u, 0);  
4     int d2 = getDepth(lca, v, 0);  
5     return d1 + d2;  
6 }
```

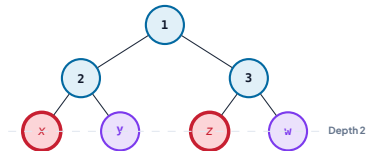
**Formula:**  $\text{dist}(u, v) = \text{dist}(\text{LCA}, u) + \text{dist}(\text{LCA}, v)$



# Variation 2: Check if Two Nodes Are Cousins

```
1 boolean isCousins(TreeNode root, int x, int y) {  
2     int[] dX = new int[1], dY = new int[1];  
3     TreeNode[] pX = new TreeNode[1], pY = new TreeNode[1];  
4     dfs(root, null, 0, x, y, dX, dY, pX, pY);  
5     return dX[0] == dY[0] && pX[0] != pY[0];  
6 }
```

**Cousins Invariant:** Same depth  $\wedge$  Different parents

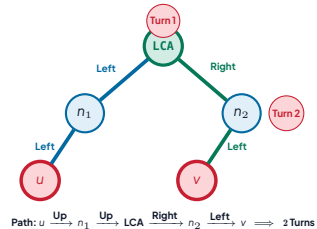


✓ Cousins:  $x$  and  $z$  ( $\text{depth}(x) = \text{depth}(z)$ ,  $\text{parent}(x) \neq \text{parent}(z)$ )

✗ Siblings:  $x$  and  $y$  ( $\text{parent}(x) == \text{parent}(y) == 2$ )

# Variation 3: Count Number of Turns Between Nodes

```
1 int countTurns(TreeNode root, int u, int v) {  
2     TreeNode lca = LCA(root, u, v);  
3     if (lca.val == u) return count(lca, v, true);  
4     if (lca.val == v) return count(lca, u, true);  
5     // Path goes: u -> LCA -> v  
6     return count(lca.left, u, false)  
7         + count(lca.right, v, true) + 1;  
8 }
```



Invariant

A turn occurs whenever the traversal direction changes (Left  $\rightarrow$  Right or Right  $\rightarrow$  Left) along the path.

# Variation 4: Print Path Between Two Nodes

```
1 List<Integer> pathBetween(TreeNode root, int u, int v) {
2     List<Integer> pU = new ArrayList<>(), pV = new ArrayList<>()
3     findPath(root, u, pU);
4     findPath(root, v, pV);
5     int i = 0;
6     while (i < pU.size() && i < pV.size() &&
7            pU.get(i).equals(pV.get(i))) i++;
8     // Combine pU[last..i-1] reversed + pV[i-1..last]
9     List<Integer> res = new ArrayList<>();
10    for (int j = pU.size() - 1; j >= i - 1; j--) res.add(pU.get(j));
11    for (int j = i; j < pV.size(); j++) res.add(pV.get(j));
12    return res;
13 }
```

## Path Merging via LCA

- Path from Root  $\rightarrow u$ : [3, 5, 2, 4]
- Path from Root  $\rightarrow v$ : [3, 5, 6]
- Common prefix matches up to index  $i - 1$  (Node 5, the LCA).
- Result: [4, 2, 5] (reversed  $u \rightarrow \text{LCA}$ ) + [6] ( $\text{LCA} \rightarrow v$ )  
 $\Rightarrow$  [4, 2, 5, 6].

# Variation 5: Maximum Node Value on Path

```
1 int maxOnPath(TreeNode root, int u, int v) {  
2     TreeNode lca = LCA(root, u, v);  
3     int maxU = getMaxToAncestor(lca, u);  
4     int maxV = getMaxToAncestor(lca, v);  
5     return Math.max(lca.val, Math.max(maxU, maxV));  
6 }
```

**Decomposition:**  $\max(u \rightarrow \text{LCA}) \cup \max(\text{LCA} \rightarrow v)$

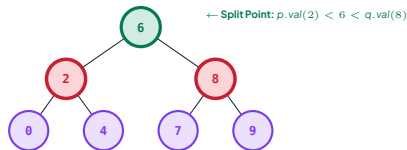
## Monotonic Path Query

Because tree paths cannot have cycles:

- Split query into two simple downward ancestor queries from  $\text{LCA} \rightarrow u$  and  $\text{LCA} \rightarrow v$ .
- Compute max on each downward branch in  $\mathcal{O}(H)$  time.

## Variation 6: LCA in a Binary Search Tree (BST)

```
1 TreeNode LCA_BST(TreeNode root, TreeNode p, TreeNode q) {  
2     if (p.val < root.val && q.val < root.val)  
3         return LCA_BST(root.left, p, q);  
4     if (p.val > root.val && q.val > root.val)  
5         return LCA_BST(root.right, p, q);  
6     return root; // SPLIT: opposite sides!  
7 }
```



In BST, when  $p$  and  $q$  lie on opposite sides of root, root is GUARANTEED to be LCA!

Invariant

In a BST, the first node whose value lies between  $p$  and  $q$  ( $p \leq \text{root} \leq q$ ) is GUARANTEED to be their LCA!



## Variation 7: Check If Node $X$ Lies on Path $(U, V)$

```
1 boolean isOnPath(TreeNode root, int u, int v, int x) {  
2     int dUV = dist(root, u, v);  
3     int dUX = dist(root, u, x);  
4     int dXV = dist(root, x, v);  
5     return dUX + dXV == dUV; // TRIANGLE EQUALITY  
6 }
```

**Geometric Invariant:**

$$\text{dist}(u, x) + \text{dist}(x, v) == \text{dist}(u, v)$$

### Tree Metric Property

Because trees are acyclic graphs with unique simple paths between any vertex pair:

- Node  $X$  lies on the path  $U \leftrightarrow V$  if and only if no detour is required to visit  $X$ .
- Total time:  $3 \times \text{LCA} = \mathcal{O}(N)$ .

# Variation 8: LCA of All Deepest Leaves

```
1 class Result { int depth; TreeNode node; }
2 Result dfs(TreeNode root) {
3     if (root == null) return new Result(0, null);
4     Result l = dfs(root.left), r = dfs(root.right);
5     if (l.depth == r.depth)
6         return new Result(l.depth + 1, root); // SPLIT OF DEEPEST LEAVES
7     return new Result(Math.max(l.depth, r.depth) + 1,
8                         l.depth > r.depth ? l.node : r.node);
9 }
```

## Deepest Leaf Split

- If `left.depth == right.depth`, the deepest leaves are spread across both subtrees  $\implies$  root is their LCA!
- If depths differ, propagate the deeper subtree's result upwards.

# Variation 9: LCA of $K$ Nodes in Binary Tree

```
1 TreeNode LCA_K(TreeNode root, Set<TreeNode> set) {  
2     if (root == null || set.contains(root)) return root;  
3     TreeNode l = LCA_K(root.left, set);  
4     TreeNode r = LCA_K(root.right, set);  
5     if (l != null && r != null) return root; // SPLIT  
6     return l != null ? l : r;  
7 }
```

**Set Inclusion:** Replaces  $p, q$  comparison with  
`set.contains(root)`

## Multi-Target Generalization

- The 4-line LCA algorithm naturally extends to  $K$  nodes.
- As soon as targets in different subtrees return non-null, the convergence node becomes the common ancestor of all subset elements below it.

# Variation 10: Step-by-Step Directions (LeetCode 2096)

```
1 String getDirections(TreeNode root, int sVal, int dVal) {
2     StringBuilder sP = new StringBuilder(), dP = new StringBuilder();
3     find(root, sVal, sP);
4     find(root, dVal, dP);
5     int i = 0;
6     while (i < sP.length() && i < dP.length() &&
7           sP.charAt(i) == dP.charAt(i)) i++;
8     return "U".repeat(sP.length() - i) + dP.substring(i);
9 }
```

Direction Transform: "U" for climb + "L"/"R" for descent

## String Reduction via LCA

- Moving from S → LCA always climbs upward  $\implies$  all steps become 'U'.
- Moving from LCA → D uses the remaining suffix of the downward path ('L'/'R').

# Summary: Master LCA & Path Decision Matrix

## Matching LCA Invariants to Path Query Problems

| Problem               | LCA Role             | Path Decomposition Formula                                                                                      | Complexity                                    |
|-----------------------|----------------------|-----------------------------------------------------------------------------------------------------------------|-----------------------------------------------|
| 1. Node Distance      | Split pivot          | $\text{dist}(\text{LCA}, u) + \text{dist}(\text{LCA}, v)$                                                       | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 2. Cousins Check      | Ancestor filter      | $\text{depth}(u) == \text{depth}(v) \wedge \text{LCA} \neq \text{parent}$                                       | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 3. Count Turns        | Direction inflection | $\text{turns}(u \rightarrow \text{LCA}) + \text{turns}(\text{LCA} \rightarrow v) + [1 \text{ if turn}]$         | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 4. Print Path         | Merging point        | $\text{reverse}(\text{path}(u \rightarrow \text{LCA})) + \text{path}(\text{LCA} \rightarrow v)$                 | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 5. Max on Path        | Sub-path splitter    | $\max(\max(u \rightarrow \text{LCA}), \max(v \rightarrow \text{LCA}), \text{LCA.val})$                          | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 6. BST LCA            | Value bracket        | $p.\text{val} \leq \text{root.val} \leq q.\text{val} \implies \text{root}$                                      | $\mathcal{O}(H)$ time, $\mathcal{O}(1)$ space |
| 7. Node on Path       | Metric check         | $\text{dist}(u, x) + \text{dist}(x, v) == \text{dist}(u, v)$                                                    | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 8. Deepest Leaves LCA | Subtree balance      | $\text{left.depth} == \text{right.depth} \implies \text{root}$                                                  | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 9. LCA of K Nodes     | Multi-target split   | $\text{targets.contains}(\text{root}) \vee (\text{left} \neq \text{null} \wedge \text{right} \neq \text{null})$ | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |
| 10. Step Directions   | Up/Down separator    | <code>"U".repeat(len_s - i) + dPath.substring(i)</code>                                                         | $\mathcal{O}(N)$ time, $\mathcal{O}(H)$ space |

Invariant

Any query on the unique path between two tree nodes decomposes into two monotonic ancestor paths via the LCA!

## Idea 5

# Root-to-Leaf Paths & Global Tree Geodesics

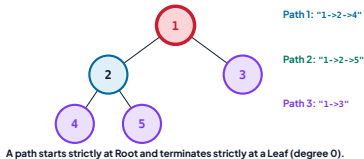
Top-Down Backtracking Paths, Tree Diameters & Leaf-to-Leaf  
Extremum Invariants

# The Root-to-Leaf Path Model: Top-Down Backtracking

## What is a Root-to-Leaf Path?

A sequence of connected vertices  $(v_0, v_1, \dots, v_k)$  satisfying two strict boundary conditions:

- **Start:**  $v_0 = \text{Root (depth 0)}$ .
- **Termination:**  $v_k$  is a **Leaf** ( $\text{left} = \text{null} \wedge \text{right} = \text{null}$ ).
- **The State Invariant:** Accumulate path state on descent; validate state *only* at leaf nodes; pop/backtrack on retreat.



Invariant

Never validate root-to-leaf paths on null! A leaf has degree 0 ( $\text{left} == \text{null} \wedge \text{right} == \text{null}$ ).

# Foundational Blueprint 1: All Root-to-Leaf Paths

```
1 void dfs(TreeNode root, String path, List<String> res) {  
2     if (root == null) return;  
3     path += root.val;  
4     if (root.left == null && root.right == null) {  
5         res.add(path); // AT LEAF: COMMIT  
6         return;  
7     }  
8     dfs(root.left, path + "->", res);  
9     dfs(root.right, path + "->", res);  
10 }
```

## State Passing Principles

1. **Preorder Append:** Append current node value as soon as visited.
2. **Leaf Invariant:** Check `left == null && right == null`. Adding here prevents duplicate path records.
3. **Immutable vs. Mutable:**
  - Strings create implicit branch snapshots.
  - Lists require explicit backtracking:  
`path.remove(path.size() - 1).`

Invariant

Top-down state accumulation reaches leaves pristine; recursion unwinding restores the parent's pristine state.



# Execution Trace: Root-to-Leaf State Transitions

## Step 1: Enter Root (1)

**Stack:** dfs(1)

**Path:** "1"

Not leaf  $\implies$  recurse Left.

Pass "1->".

## Step 2: Enter Left (2)

**Stack:** dfs(2)

**Path:** "1->2"

Not leaf  $\implies$  recurse Left to (4).

Pass "1->2->".

## Step 3: Leaf (4) Hit

**Stack:** dfs(4)

**Path:** "1->2->4"

**Leaf detected!**

$\implies$  res.add("1->2->4").

Pop frame.

## Step 4: Leaf (5) & (3)

**Branch 5:** "1->2->5"

**Branch 3:** "1->3"

Backtrack restores prefix.

**Result:** 3 paths collected.

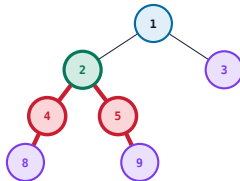
**Backtracking Principle:** Every branch explores independently without state contamination from previous sibling traversals.

# Global Tree Geodesics: The Tree Diameter Mindset

## What is Tree Diameter?

The **Diameter** (longest path) of a binary tree is the maximum number of edges between *any two nodes*:

- The path may or may not pass through the root!
- Every simple path in a tree has a unique **highest apex node** where it reaches maximum height and bends.
- **Global Search**: We must compute the longest split path at every node as candidate apex!



Diameter does NOT pass through Root (1)!

Longest path bridges through Node 2 (len = 4).

Invariant

A tree path can turn at most ONCE. The turning point is the local LCA connecting its two branch endpoints.

# Foundational Blueprint 2: The Split vs. Return Invariant

```
1 int diameter = 0;
2 int maxDepth(TreeNode root) {
3     if (root == null) return 0;
4     int left = maxDepth(root.left);
5     int right = maxDepth(root.right);
6     diameter = Math.max(diameter, left + right); // SPLIT
7     return Math.max(left, right) + 1;           // RETURN
8 }
```

## The Split vs. Return Rule

1. **Local Split Candidate:** The path bending at node  $u$  combines both branches:  $\text{length} = L + R$ . We update the global maximum.
2. **Return to Parent:** A path continuing upward to  $u$ 's parent cannot branch twice! It must choose the **single deepest branch**:  $\max(L, R) + 1$ .

Invariant

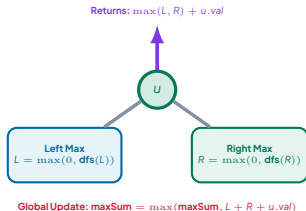
Split locally ( $L + R$ ) to test global optimality; return linearly ( $\max(L, R) + 1$ ) to preserve path validity.

# Maximum Path Sum: Node-to-Node vs. Leaf-to-Leaf

## 1. Any Node to Any Node (LC 124)

Rule: Path can start and end at any node.

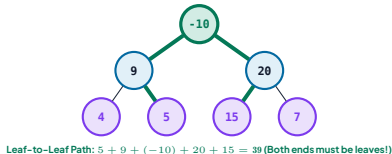
- **Negative Pruning:** If subtree sum  $< 0$ , prune it:  
 $L = \max(0, \text{dfs}(L))$ .
- **Split:**  $\text{maxSum} = \max(\text{maxSum}, L + R + u.\text{val})$ .
- **Return:**  $\max(L, R) + u.\text{val}$ .



## 2. Strictly Leaf to Leaf

Rule: Both endpoints must be genuine leaves!

- **No Pruning:** Cannot discard negative leaves.
- **1 Child:** Path **cannot** turn at node with 1 child; must pass through to parent!
- **2 Children:** Split only if both subtrees exist.



# Complexity Analysis: Paths, Diameters & Geodesics

## Time Complexity: $\mathcal{O}(N)$

- Bottom-up postorder visits each node exactly once.
- Top-down backtracking visits each edge at most twice (forward + backtrack).
- Copying leaf paths takes  $\mathcal{O}(H)$  at each leaf, giving total  $\mathcal{O}(N \cdot H)$  string copying worst-case.

## Space Complexity: $\mathcal{O}(H)$

- Recursion stack depth is strictly bounded by tree height  $H$ .
- **Balanced Tree:**  $H = \mathcal{O}(\log N)$  stack frames.
- **Skewed Tree:**  $H = \mathcal{O}(N)$  stack frames.
- Backtracking list maintains current branch of length  $\leq H$ .

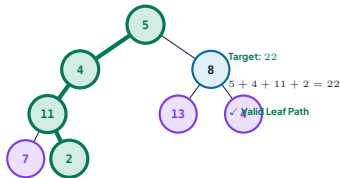
**Invariant**

**Tree geodesics compute global optimums in optimal  $\mathcal{O}(N)$  time by combining local split updates with single-branch returns.**

# Variation 1: Path Sum I (Target Sum Existence)

```
1 boolean hasPathSum(TreeNode root, int targetSum) {  
2     if (root == null) return false;  
3     if (root.left == null && root.right == null)  
4         return targetSum == root.val;  
5     return hasPathSum(root.left, targetSum - root.val) ||  
6         hasPathSum(root.right, targetSum - root.val);  
7 }
```

**Invariant:** Subtract on descent; check equality strictly at leaf



## Variation 2: Path Sum II (All Target Paths)

```
1 void dfs(TreeNode root, int sum, List<Integer> path,  
2     List<List<Integer>> res) {  
3     if (root == null) return;  
4     path.add(root.val);  
5     if (root.left == null && root.right == null  
6         && sum == root.val) {  
7         res.add(new ArrayList<>(path)); // COPY  
8     }  
9     dfs(root.left, sum - root.val, path, res);  
10    dfs(root.right, sum - root.val, path, res);  
11    path.remove(path.size() - 1); // POP  
12 }
```

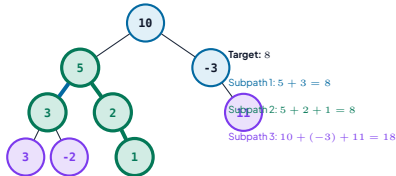
**Backtracking:** path.add() on enter, path.remove() on exit

### Step Deduction – Memory Invariant

Using a single shared mutable List avoids  $\mathcal{O}(N^2)$  memory copying on non-leaf nodes. We deep-copy *only* upon hitting valid target leaves.

## Variation 3: Path Sum III (Subpath Prefix Sums)

```
1 int dfs(TreeNode root, long sum, int target,
2     Map<Long, Integer> pre) {
3     if (root == null) return 0;
4     sum += root.val;
5     int count = pre.getDefault(sum - target, 0);
6     pre.put(sum, pre.getOrDefault(sum, 0) + 1);
7     count += dfs(root.left, sum, target, pre)
8             + dfs(root.right, sum, target, pre);
9     pre.put(sum, pre.get(sum) - 1); // BACKTRACK
10    return count;
11 }
```



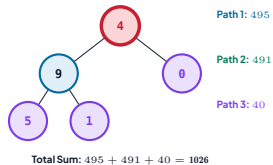
Subarray  $\rightarrow$  Tree:  $\mathcal{O}(N)$  via running prefix sum map



# Variation 4: Sum Root-to-Leaf Numbers

```
1 int sumNumbers(TreeNode root, int curr) {  
2     if (root == null) return 0;  
3     curr = curr * 10 + root.val;  
4     if (root.left == null && root.right == null)  
5         return curr;  
6     return sumNumbers(root.left, curr) +  
7           sumNumbers(root.right, curr);  
8 }
```

**Base-10 Invariant:** Shift decimal:  $\text{curr} \times 10 + \text{val}$



## Variation 5: Smallest String Starting From Leaf

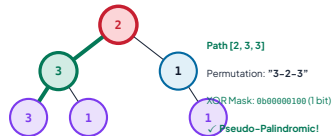
```
1 String smallest = "~";
2 void dfs(TreeNode root, StringBuilder sb) {
3     if (root == null) return;
4     sb.append((char)('a' + root.val));
5     if (root.left == null && root.right == null) {
6         String path = sb.reverse().toString();
7         if (path.compareTo(smallest) < 0) smallest = path;
8         sb.reverse(); // RESTORE ORDER
9     }
10    dfs(root.left, sb); dfs(root.right, sb);
11    sb.setLength(sb.length() - 1);
12 }
```

### Step Deduction – Lexicographical Traversal

Because string order is evaluated from leaf to root, greedy top-down choice fails. We accumulate to genuine leaves, reverse, and update the global minimum string.

# Variation 6: Pseudo-Palindromic Paths

```
1 int dfs(TreeNode root, int mask) {
2     if (root == null) return 0;
3     mask ^= (1 << root.val); // FLIP PARITY BIT
4     if (root.left == null && root.right == null) {
5         return (mask & (mask - 1)) == 0 ? 1 : 0;
6     }
7     return dfs(root.left, mask) + dfs(root.right, mask);
8 }
```



Bitmask Invariant:  $\leq$  1 odd digit  
 $\iff (\text{mask} \& (\text{mask} - 1)) == 0$

# Variation 7: Binary Tree Maximum Path Sum

```
1 int maxSum = Integer.MIN_VALUE;
2 int maxGain(TreeNode root) {
3     if (root == null) return 0;
4     int L = Math.max(0, maxGain(root.left));
5     int R = Math.max(0, maxGain(root.right));
6     maxSum = Math.max(maxSum, L + R + root.val);
7     return Math.max(L, R) + root.val;
8 }
```

**Postorder Bubble:** Prune negative branches with  
`max(0, gain)`

## Step Deduction – The Zero-Clamp Invariant

If a child subtree yields negative sum, returning 0 effectively drops that branch from the apex path. This handles trees with negative node values optimally.

# Variation 8: Maximum Path Sum Between Two Leaves

```
1 int maxLeafSum(TreeNode root, int[] res) {  
2     if (root == null) return 0;  
3     if (root.left == null && root.right == null)  
4         return root.val;  
5     int L = maxLeafSum(root.left, res);  
6     int R = maxLeafSum(root.right, res);  
7     if (root.left != null && root.right != null) {  
8         res[0] = Math.max(res[0], L + R + root.val);  
9         return Math.max(L, R) + root.val;  
10    }  
11    return root.left != null ? L + root.val  
12                               : R + root.val;  
13 }
```

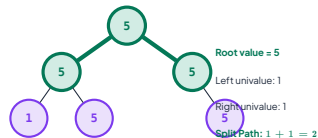
## Step Deduction – Leaf-to-Leaf Constraint

Unlike LC 124, a single leaf cannot form a path alone. If a node lacks one child, the path cannot apex here and must pass through upward.

**Single Child Rule:** Cannot bend at degree 1; must pass through

# Variation 9: Longest Univalue Path

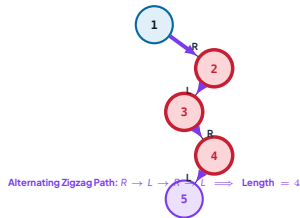
```
1 int maxlen = 0;
2 int arrow(TreeNode root) {
3     if (root == null) return 0;
4     int L = arrow(root.left), R = arrow(root.right);
5     int lA = (root.left != null &&
6             root.left.val == root.val) ? L + 1 : 0;
7     int rA = (root.right != null &&
8             root.right.val == root.val) ? R + 1 : 0;
9     maxlen = Math.max(maxlen, lA + rA);
10    return Math.max(lA, rA);
11 }
```



**Univalue Matching:** Extend length by +1 only if child equals root

# Variation 10: Longest Zigzag Path

```
1 int maxZigZag = 0;
2 void dfs(TreeNode root, boolean isLeft, int steps) {
3     if (root == null) return;
4     maxZigZag = Math.max(maxZigZag, steps);
5     if (isLeft) {
6         dfs(root.left, false, steps + 1); // R->L
7         dfs(root.right, true, 1);          // RESET
8     } else {
9         dfs(root.right, true, steps + 1); // L->R
10        dfs(root.left, false, 1);         // RESET
11    }
12 }
```



# Master Decision Matrix: Paths & Geodesics

## Comparison of 10 Path & Geodesic Variations

| Variation              | Direction | Branch State / Return                 | Split Rule at Apex                       | Time / Space                    |
|------------------------|-----------|---------------------------------------|------------------------------------------|---------------------------------|
| 1. Path Sum I          | Top-Down  | Remaining sum - val                   | Boolean OR at leaf                       | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 2. Path Sum II         | Top-Down  | Mutable List<int> path                | Deep-copy on valid leaf                  | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 3. Path Sum III        | Top-Down  | Prefix Sum Frequency Map              | Subarray sum lookup                      | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 4. Sum Root-Leaf       | Top-Down  | Decimal curr $\times 10 + \text{val}$ | Sum leaves: $L + R$                      | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 5. Smallest String     | Top-Down  | Reverse string buffer                 | String lexicographical min               | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 6. Pseudo-Palindromic  | Top-Down  | Bitmask XOR parity                    | Single bit: $m \& (m - 1) == 0$          | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 7. Max Path Sum (Node) | Bottom-Up | $\max(0, \text{dfs}) + \text{val}$    | Split: $L + R + \text{val}$              | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 8. Max Path Sum (Leaf) | Bottom-Up | Pass through if degree 1              | Split only if 2 children                 | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 9. Longest Univalue    | Bottom-Up | Length if child equals root           | Split: $L_{\text{uni}} + R_{\text{uni}}$ | $\mathcal{O}(N)/\mathcal{O}(H)$ |
| 10. Longest Zigzag     | Top-Down  | Direction flag + steps                | Toggle +1 vs Reset 1                     | $\mathcal{O}(N)/\mathcal{O}(H)$ |



## Idea 6

# Binary Search Trees (BST)

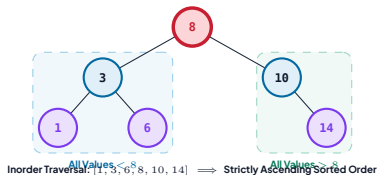
The Sorted Inorder Invariant, Search-Insert-Delete Mechanics & 10 Essential Interview Types

# The Binary Search Tree Mindset: Ordered Hierarchy

## What is a Binary Search Tree?

A binary tree is a **BST** if for every node  $u$ :

- $\forall x \in \text{LeftSubtree}(u) : x.\text{val} < u.\text{val}$
- $\forall y \in \text{RightSubtree}(u) : y.\text{val} > u.\text{val}$
- Both left and right subtrees must also be valid BSTs.
- **Fundamental Invariant:** Inorder Traversal of a BST produces a *strictly ascending sorted array*!



**Invariant**

The BST condition is **GLOBAL**: every single node in the left subtree must be smaller than the ancestor root.

# Foundational Operation 1: Search in a BST

```
1 TreeNode searchBST(TreeNode root, int val) {  
2     if (root == null || root.val == val)  
3         return root;  
4     if (val < root.val)  
5         return searchBST(root.left, val);  
6     return searchBST(root.right, val);  
7 }
```

**Search Invariant:** Prune half the tree at each step:  
 $\mathcal{O}(H)$  time

**Invariant**

Search in BST mimics binary search on a sorted array, reducing candidate nodes by 1 branch per step.

## Binary Search on Trees

1. **Match:** If  $val == root.val$ , key found.
2. **Left Branch:** If  $val < root.val$ , right subtree cannot contain  $val \implies$  discard right!
3. **Right Branch:** If  $val > root.val$ , left subtree cannot contain  $val \implies$  discard left!
4. **Iterative Equivalent:** Can be written in  $\mathcal{O}(1)$  space using a simple while loop.

# Foundational Operation 2: Insert into a BST

```
1 TreeNode insertIntoBST(TreeNode root, int val) {  
2     if (root == null) return new TreeNode(val);  
3     if (val < root.val)  
4         root.left = insertIntoBST(root.left, val);  
5     else  
6         root.right = insertIntoBST(root.right, val);  
7     return root;  
8 }
```

## Insertion Invariants

- **Leaf Insertion:** A new key is always inserted as a brand new **Leaf** at depth  $H$ .
- **Unique Path:** There is exactly ONE valid leaf location in the BST for any unique key.
- **Time:**  $\mathcal{O}(H)$  where  $H$  is the tree height.

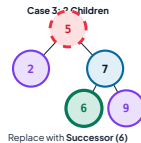
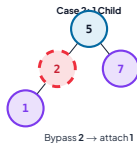
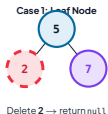
**Pointer Reconstruction:** Re-link child pointers on unwinding

**Invariant**

Inserting a node never requires restructuring existing vertices; traverse to the null gap and attach.

# Foundational Operation 3: Delete (The 3 Cases)

## Three Structural Cases in BST Node Deletion



1. **Case 1 (Leaf):** Degree 0  $\implies$  directly return null to parent.
2. **Case 2 (1 Child):** Degree 1  $\implies$  bypass deleted node by returning its non-null child.
3. **Case 3 (2 Children):** Replace value with **Inorder Successor** ( $\min(\text{Right})$ ), then recursively delete that successor from right subtree.

# Foundational Blueprint 3: Delete in Java

```
1 TreeNode deleteNode(TreeNode root, int key) {  
2     if (root == null) return null;  
3     if (key < root.val)  
4         root.left = deleteNode(root.left, key);  
5     else if (key > root.val)  
6         root.right = deleteNode(root.right, key);  
7     else {  
8         if (root.left == null) return root.right;  
9         if (root.right == null) return root.left;  
10        TreeNode succ = findMin(root.right);  
11        root.val = succ.val;  
12        root.right = deleteNode(root.right, succ.val);  
13    }  
14    return root;  
15 }
```

## Why Inorder Successor?

- $\text{min}(\text{Right})$  is guaranteed to have **no left child** (at most degree 1).
- Deleting the successor from the right subtree falls strictly into **Case 1 or Case 2**.
- Inorder Predecessor ( $\text{max}(\text{Left})$ ) works equally well symmetrically.

**Invariant**

Inorder successor maintains BST ordering perfectly because it is strictly greater than all left nodes and smaller than all right nodes.

# BST Complexity Analysis & The Skewing Hazard

## Balanced BST ( $\mathcal{O}(\log N)$ )

- When insertions maintain balanced height:  $H = \mathcal{O}(\log N)$ .
- **Search, Insert, Delete:**  $\mathcal{O}(\log N)$  time.
- Provides logarithmic bounds for dynamic sets and priority ranges.

## Skewed BST Hazard ( $\mathcal{O}(N)$ )

- Inserting sorted keys  $[1, 2, 3, 4, 5]$  degrades BST into a linear linked list.
- Height collapses to  $H = N$ .
- Search, Insert, Delete degrade to  $\mathcal{O}(N)$ .
- **Remedy:** Self-balancing trees (AVL / Red-Black Trees).

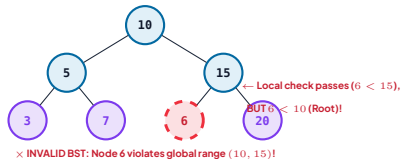
Invariant

Standard BST operations are strictly  $\mathcal{O}(H)$ . Self-balancing trees maintain  $H = \mathcal{O}(\log N)$  via tree rotations.

# Type 1: Validate Binary Search Tree

```
1 boolean isValidBST(TreeNode root) {  
2     return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);  
3 }  
4 boolean validate(TreeNode node, long min, long max) {  
5     if (node == null) return true;  
6     if (node.val <= min || node.val >= max) return false;  
7     return validate(node.left, min, node.val) &&  
8         validate(node.right, node.val, max);  
9 }
```

**Global Range Invariant:** Subtree bounds shrink on descent: (min, max)





## Type 2: $K$ -th Smallest Element in a BST

```
1 int count = 0, res = -1;
2 void inorder(TreeNode root, int k) {
3     if (root == null) return;
4     inorder(root.left, k);
5     if (++count == k) {
6         res = root.val;
7         return;
8     }
9     inorder(root.right, k);
10 }
```

**Inorder Monotonicity:** Inorder visits keys in sorted order;  $K$ -th visit is answer

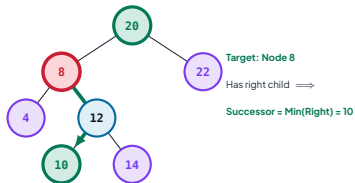
### Step Deduction – Optimal Stopping

Because Inorder traversal is strictly sorted, stopping as soon as  $\text{count} == k$  yields optimal  $\mathcal{O}(H + K)$  time without visiting the rest of the tree.

# Type 3: Inorder Successor in BST

```
1  TreeNode inorderSuccessor(TreeNode root, TreeNode p) {  
2      TreeNode succ = null;  
3      while (root != null) {  
4          if (p.val < root.val) {  
5              succ = root;    // POTENTIAL SUCCESSOR  
6              root = root.left; // GO LEFT  
7          } else {  
8              root = root.right; // GO RIGHT  
9          }  
10     }  
11     return succ;  
12 }
```

**Turning Invariant:** Last node where we turned Left is the successor



# Type 4: Binary Search Tree Iterator

```
1 Deque<TreeNode> stack = new ArrayDeque<>();
2 public BSTIterator(TreeNode root) { pushLeft(root); }
3 public int next() {
4     TreeNode top = stack.pop();
5     pushLeft(top.right); // REFILL RIGHT BRANCH
6     return top.val;
7 }
8 public boolean hasNext() { return !stack.isEmpty(); }
9 private void pushLeft(TreeNode node) {
10     while (node != null) {
11         stack.push(node);
12         node = node.left;
13     }
14 }
```

## Controlled DFS Invariant

- **Amortized**  $\mathcal{O}(1)$ : Each of the  $N$  nodes is pushed and popped exactly once across all `next()` calls.
- **Space**  $\mathcal{O}(H)$ : Stack depth never exceeds tree height  $H$ .
- Simulates lazy stream evaluation over massive BSTs.

# Type 5: Convert Sorted Array to Balanced BST

```
1  TreeNode sortedArrayToBST(int[] nums) {  
2      return build(nums, 0, nums.length - 1);  
3  }  
4  TreeNode build(int[] nums, int L, int R) {  
5      if (L > R) return null;  
6      int mid = L + (R - L) / 2;  
7      TreeNode root = new TreeNode(nums[mid]);  
8      root.left = build(nums, L, mid - 1);  
9      root.right = build(nums, mid + 1, R);  
10     return root;  
11 }
```

**Divide & Conquer:** Pick middle element as root to guarantee  $H = \mathcal{O}(\log N)$

## Step Deduction – Optimal Height Invariant

Choosing the median element partitions the remaining  $N - 1$  elements into two sub-arrays of nearly equal size ( $\pm 1$ ), guaranteeing minimal possible tree height.

# Type 6: Construct BST from Preorder

```
1 int idx = 0;
2 TreeNode bstFromPreorder(int[] pre) {
3     return build(pre, Integer.MAX_VALUE);
4 }
5 TreeNode build(int[] pre, int bound) {
6     if (idx == pre.length || pre[idx] > bound)
7         return null;
8     TreeNode root = new TreeNode(pre[idx++]);
9     root.left = build(pre, root.val); // L < root.val
10    root.right = build(pre, bound);   // R < bound
11    return root;
12 }
```

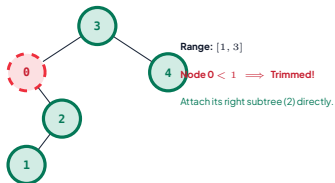
**Upper Bound Invariant:** Construct in single  $\mathcal{O}(N)$  pass without sorting

## Step Deduction – Linear Preorder Construction

Instead of sorting ( $\mathcal{O}(N \log N)$ ) or searching split point ( $\mathcal{O}(N^2)$ ), propagating upper bounds parses tokens in exact preorder arrival order in  $\mathcal{O}(N)$  time.

# Type 7: Trim a Binary Search Tree

```
1 TreeNode trimBST(TreeNode root, int low, int high) {  
2     if (root == null) return null;  
3     if (root.val < low)  
4         return trimBST(root.right, low, high);  
5     if (root.val > high)  
6         return trimBST(root.left, low, high);  
7     root.left = trimBST(root.left, low, high);  
8     root.right = trimBST(root.right, low, high);  
9     return root;  
10 }
```



**Pruning Invariant:** If node < low, entire left subtree is discarded

# Type 8: Range Sum of BST

```
1 int rangeSumBST(TreeNode root, int low, int high) {  
2     if (root == null) return 0;  
3     if (root.val < low)  
4         return rangeSumBST(root.right, low, high);  
5     if (root.val > high)  
6         return rangeSumBST(root.left, low, high);  
7     return root.val +  
8         rangeSumBST(root.left, low, high) +  
9         rangeSumBST(root.right, low, high);  
10 }
```

**Selective Traversal:** Skip branches outside  $[low, high]$

## Step Deduction – Range Optimization

By testing `root.val` against range boundaries, we prune irrelevant branches completely, running proportional to keys inside the range ( $\mathcal{O}(H + K)$ ) rather than full tree  $\mathcal{O}(N)$ .

# Type 9: Two Sum IV - Input is a BST

```
1 boolean findTarget(TreeNode root, int k) {  
2     Set<Integer> set = new HashSet<>();  
3     return dfs(root, k, set);  
4 }  
5 boolean dfs(TreeNode root, int k, Set<Integer> set) {  
6     if (root == null) return false;  
7     if (set.contains(k - root.val)) return true;  
8     set.add(root.val);  
9     return dfs(root.left, k, set) ||  
10         dfs(root.right, k, set);  
11 }
```

**Dual Approaches:** (1) HashSet  $\mathcal{O}(N)$  or (2) Dual BST Iterators  $\mathcal{O}(H)$  space

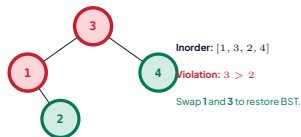
## Step Deduction – Two-Pointer Invariant

Using two BST Iterators (forward from min and reverse from max) allows running standard two-pointer convergence in  $\mathcal{O}(N)$  time with strictly  $\mathcal{O}(H)$  memory.



# Type 10: Recover Binary Search Tree

```
1  TreeNode first = null, second = null, prev = null;
2  void inorder(TreeNode root) {
3      if (root == null) return;
4      inorder(root.left);
5      if (prev != null && prev.val >= root.val) {
6          if (first == null) first = prev; // 1ST DROP
7          second = root;                  // 2ND DROP
8      }
9      prev = root;
10     inorder(root.right);
11 }
```



**Invariant**

In a sorted array with two swapped elements, the first swapped element is the larger element of the 1st inversion, and the second is the smaller element of the 2nd inversion.

# Master Decision Matrix: BST Problem Taxonomy

## Taxonomy of 10 Master Binary Search Tree Problems

| Problem Type       | Core Technique                | Inorder Invariant Utilized     | Branch Pruning Rule                              | Time / Space                         |
|--------------------|-------------------------------|--------------------------------|--------------------------------------------------|--------------------------------------|
| 1. Validate BST    | Range Bounding ( $min, max$ ) | Global inequality validation   | Prune if $val \leq min \vee val \geq max$        | $\mathcal{O}(N)/\mathcal{O}(H)$      |
| 2. K-th Smallest   | Inorder Counter               | Sorted monotonic sequence      | Stop immediately at count == K                   | $\mathcal{O}(H + K)/\mathcal{O}(H)$  |
| 3. Successor       | Iterative Pointer Tracking    | Next larger element in order   | Record last Left turn                            | $\mathcal{O}(H)/\mathcal{O}(1)$      |
| 4. BST Iterator    | Controlled Stack DFS          | Lazy sorted stream generator   | Amortized $\mathcal{O}(1)$ next element          | $\mathcal{O}(1)/\mathcal{O}(H)$      |
| 5. Array to BST    | Divide & Conquer (Median)     | Subarray mid becomes root      | Equal size left/right partition                  | $\mathcal{O}(N)/\mathcal{O}(\log N)$ |
| 6. Preorder to BST | Upper Bound Parsing           | Monotonic boundary check       | If $val > bound \implies$ return null            | $\mathcal{O}(N)/\mathcal{O}(H)$      |
| 7. Trim BST        | Range Pruning                 | Keep keys within $[low, high]$ | If $val < low$ , bypass to right subtree         | $\mathcal{O}(N)/\mathcal{O}(H)$      |
| 8. Range Sum       | Branch Elimination            | Accumulate in-range keys       | Skip left if $val < low$ , right if $val > high$ | $\mathcal{O}(H + K)/\mathcal{O}(H)$  |
| 9. Two Sum IV      | Hash Set / Dual Iterators     | Sorted convergence pointers    | Standard two-pointer shrinkage                   | $\mathcal{O}(N)/\mathcal{O}(H)$      |
| 10. Recover BST    | Inversion Detection           | Exactly 2 inorder drops        | Swap first ( $max$ ) and second ( $min$ )        | $\mathcal{O}(N)/\mathcal{O}(H)$      |

## Idea 7

# Heaps & Priority Queues

Array-Based Binary Heaps, Sift-Down Invariants & The Top-K Mental Model

# The Binary Heap Mindset: Array-Backed Complete Trees

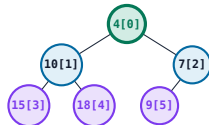
## What is a Binary Heap?

A **Complete Binary Tree** packed densely into a 1D array without pointers:

- **0-Indexed Arithmetic:**
  - $\text{Left}(i) = 2i + 1$     $\text{Right}(i) = 2i + 2$
  - $\text{Parent}(i) = \lfloor (i - 1) / 2 \rfloor$
- **Min-Heap Order:** Parent  $\leq$  Children (Root = global minimum).
- **Max-Heap Order:** Parent  $\geq$  Children (Root = global maximum).

1D Flattened Array Representation:

|     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|
| 4   | 10  | 7   | 15  | 18  | 9   |
| [0] | [1] | [2] | [3] | [4] | [5] |



**0-Indexed Arithmetic:**

$$\text{Left}(i) = 2i + 1, \text{Right}(i) = 2i + 2$$

$$\text{Parent}(i) = \lfloor (i - 1) / 2 \rfloor$$

**Invariant**

Heaps eliminate pointer overhead entirely; tree navigation is purely  $2i + 1$  and  $\lfloor (i - 1) / 2 \rfloor$  array indexing.

# Heap Mechanics: Sift Operations & $\mathcal{O}(N)$ Build-Heap

## 1. Dynamic Sift Operations

- **Insert / Sift-Up** ( $\mathcal{O}(\log N)$ ): Append new element to array end, bubble up with parent until heap order holds.
- **Poll / Sift-Down** ( $\mathcal{O}(\log N)$ ): Swap root with last leaf, shrink array by 1, sink new root down with smaller child.

## 2. Linear Build-Heap (Floyd)

Why is Build-Heap  $\mathcal{O}(N)$ , not  $\mathcal{O}(N \log N)$ ?

- Run `siftDown` from index  $\lfloor N/2 \rfloor - 1$  down to 0.
- Most nodes are near the bottom (height 0, 1), doing  $\leq 1$  swap!
- Total cost:  $\sum_{h=0}^{\log N} \frac{N}{2^{h+1}} \cdot h = N \sum \frac{h}{2^{h+1}} = \mathcal{O}(N)$ .

**Invariant**

**Bottom-up heapification is  $\mathcal{O}(N)$  because 50% of nodes are leaves requiring 0 operations.**

# How to Spot a Heap Problem: The 4 Grand Archetypes

## 1. Bounded Top-K

**Goal:** Top  $K$  extremums in unsorted stream.

**Heap:** Size  $K$  bounded heap.

$\mathcal{O}(N \log K)$  time.

## 2. Greedy Merge

**Goal:** Compress 2-by-2 to minimize total cost.

**Heap:** Pop 2 smallest, push sum.

$\mathcal{O}(N \log N)$  time.

## 3. K-Way Merge

**Goal:** Merge  $K$  sorted streams / matrix rows.

**Heap:** Store current head of  $K$  lists.

$\mathcal{O}(N \log K)$  time.

## 4. Dual Heaps

**Goal:** Continuous stream median / balance.

**Heap:** Max-Heap (low 50%) + Min-Heap (high 50%).

$\mathcal{O}(\log N)$  insert.

**Invariant**

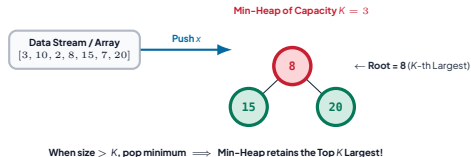
If a problem asks for continuous extremums, partial rankings, or greedy combinations, think Heap first.

# The Golden Rule: Min-Heap for Top- $K$ Largest

## The Inversion Principle

A counter-intuitive strategy for streaming inputs:

- **For  $K$ -th Largest:** Maintain a **Min-Heap** of size  $K$ .
  - Root is min of top  $K \implies$  exactly  $K$ -th largest.
  - Smaller elements evicted on overflow.
- **For  $K$ -th Smallest:** Maintain a **Max-Heap** of size  $K$ .
- **Complexity:**  $\mathcal{O}(N \log K)$  time,  $\mathcal{O}(K)$  space.



Invariant

**Counter-Intuitive Invariant:** Use Min-Heap to track largest elements, and Max-Heap for smallest.

# Variation 1: $K$ -th Largest Element in an Array

```
1 int findKthLargest(int[] nums, int k) {  
2     PriorityQueue<Integer> pq = new PriorityQueue<>();  
3     for (int x : nums) {  
4         pq.offer(x);  
5         if (pq.size() > k) pq.poll(); // EVICT MIN  
6     }  
7     return pq.peek();                // K-TH LARGEST  
8 }
```

**Bounded Min-Heap:** Size never exceeds

$K + 1 \implies \mathcal{O}(N \log K)$  time

## Step Deduction – Stream Efficiency

Compared to sorting ( $\mathcal{O}(N \log N)$ ) or full heap ( $\mathcal{O}(N)$  space), the bounded size- $K$  heap processes unbounded real-time data streams using strictly  $\mathcal{O}(K)$  memory.



## Variation 2: Top $K$ Frequent Elements

```
1 int[] topKFrequent(int[] nums, int k) {  
2     Map<Integer, Integer> map = new HashMap<>();  
3     for (int x : nums) map.put(x, map.getOrDefault(x, 0) + 1);  
4     PriorityQueue<Integer> pq = new PriorityQueue<>(  
5         (a, b) -> map.get(a) - map.get(b) // MIN-HEAP BY FREQ  
6     );  
7     for (int key : map.keySet()) {  
8         pq.offer(key);  
9         if (pq.size() > k) pq.poll();  
10    }  
11    return pq.stream().mapToInt(i -> i).toArray();  
12 }
```

### Step Deduction – Frequency Comparator

By feeding map keys into a size- $K$  Min-Heap ordered by frequency count, elements with lower frequency are evicted first, leaving the  $K$  most frequent keys.

## Variation 3: $K$ Closest Points to Origin

```
1 int[][] kClosest(int[][] points, int k) {
2     PriorityQueue<int[]> pq = new PriorityQueue<(<
3         (a, b) -> (b[0]*b[0] + b[1]*b[1]) -
4             (a[0]*a[0] + a[1]*a[1]) // MAX-HEAP
5     >);
6     for (int[] p : points) {
7         pq.offer(p);
8         if (pq.size() > k) pq.poll(); // EVICT FARTHEST
9     }
10    return pq.toArray(new int[k][]);
11 }
```

### Step Deduction – Distance Invariant

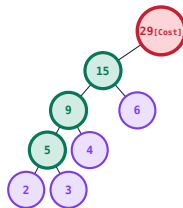
Because Euclidean distance comparison  $\sqrt{x^2 + y^2}$  is monotonic, comparing  $x^2 + y^2$  directly avoids costly square root floating point calculations.

**Max-Heap for  $K$ -Smallest:** Evicts farthest point whenever size  $> K$

# Variation 4: Connect Ropes to Minimize Cost

```
1 int minCost(int[] ropes) {  
2     PriorityQueue<Integer> pq = new PriorityQueue<>();  
3     for (int r : ropes) pq.offer(r);  
4     int totalCost = 0;  
5     while (pq.size() > 1) {  
6         int first = pq.poll(), second = pq.poll();  
7         int sum = first + second;  
8         totalCost += sum;  
9         pq.offer(sum); // PUSH MERGED BACK  
10    }  
11    return totalCost;  
12 }
```

**Greedy Choice:** Merging the 2 shortest ropes minimizes cumulative cost



Ropes: [2, 3, 4, 6]

Step 1: Merge (2, 3) → 5 (Cost 5)

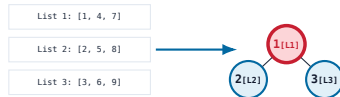
Step 2: Merge (4, 5) → 9 (Cost 9)

Step 3: Merge (6, 9) → 15 (Cost 15)

Total Cost: 5 + 9 + 15 = 29

# Variation 5: Merge $K$ Sorted Lists

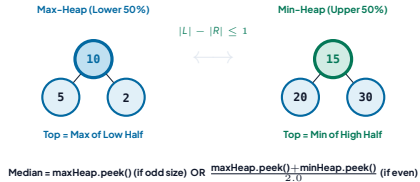
```
1 ListNode mergeKLists(ListNode[] lists) {
2     PriorityQueue<ListNode> pq = new PriorityQueue<>(
3         (a, b) -> a.val - b.val // MIN-HEAP
4     );
5     for (ListNode node : lists)
6         if (node != null) pq.offer(node);
7     ListNode dummy = new ListNode(0), curr = dummy;
8     while (!pq.isEmpty()) {
9         ListNode minNode = pq.poll();
10        curr.next = minNode; curr = curr.next;
11        if (minNode.next != null) pq.offer(minNode.next);
12    }
13    return dummy.next;
14 }
```



Always pop global minimum (1), append to merged output, and insert next node from same list (4).

# Variation 6: Find Median from Data Stream

```
1 PriorityQueue<Integer> low =  
2     new PriorityQueue<>(Collections.reverseOrder());  
3 PriorityQueue<Integer> high =  
4     new PriorityQueue<>();  
5 public void addNum(int num) {  
6     low.offer(num);  
7     high.offer(low.poll()); // FILTER THROUGH LOW  
8     if (low.size() < high.size())  
9         low.offer(high.poll()); // BALANCE SIZES  
10 }  
11 public double findMedian() {  
12     return low.size() > high.size() ?  
13         low.peek() : (low.peek() + high.peek()) / 2.0;  
14 }
```



# Variation 7: Reorganize String

```
1 String reorganizeString(String s) {
2     int[] count = new int[26];
3     for (char c : s.toCharArray()) count[c - 'a']++;
4     PriorityQueue<Character> pq =
5         new PriorityQueue<>((a, b) ->
6             count[b - 'a'] - count[a - 'a']);
7     for (int i = 0; i < 26; i++)
8         if (count[i] > 0) pq.offer((char)('a' + i));
9     StringBuilder sb = new StringBuilder();
10    Character block = null;
11    while (!pq.isEmpty()) {
12        char curr = pq.poll();
13        sb.append(curr);
14        count[curr - 'a']--;
15        if (block != null && count[block - 'a'] > 0)
16            pq.offer(block);
17        block = curr;
18    }
19    return sb.length() == s.length() ? sb.toString() : "";
20 }
```

## Step Deduction – Greedy Spacing Invariant

Always append the most frequent remaining character, but hold it in a temporary cooldown variable `block` so it cannot be chosen in consecutive turns.

# Variation 8: Task Scheduler

```
1 int leastInterval(char[] tasks, int n) {
2     int[] count = new int[26];
3     for (char c : tasks) count[c - 'A']++;
4     PriorityQueue<Integer> pq = new PriorityQueue<>(
5         Collections.reverseOrder() // MAX-HEAP
6     );
7     for (int f : count) if (f > 0) pq.offer(f);
8     int cycles = 0;
9     while (!pq.isEmpty()) {
10         List<Integer> temp = new ArrayList<>();
11         for (int i = 0; i <= n; i++) {
12             if (!pq.isEmpty()) temp.add(pq.poll());
13         }
14         for (int f : temp) if (--f > 0) pq.offer(f);
15         cycles += pq.isEmpty() ? temp.size() : n + 1;
16     }
17     return cycles;
18 }
```

## Step Deduction – Round-Robin Epochs

In each epoch of length  $n + 1$ , schedule the highest frequency tasks first. If fewer tasks exist than  $n + 1$ , remaining slots become CPU idle cycles.

## Variation 9: $K$ -th Smallest in Sorted Matrix

```
1 int kthSmallest(int[][] matrix, int k) {
2     int n = matrix.length;
3     PriorityQueue<int[]> pq = new PriorityQueue<>((
4         (a, b) -> a[0] - b[0] // MIN-HEAP: {val, r, c}
5     ));
6     for (int r = 0; r < Math.min(n, k); r++)
7         pq.offer(new int[]{matrix[r][0], r, 0});
8     for (int i = 0; i < k - 1; i++) {
9         int[] top = pq.poll();
10        int r = top[1], c = top[2];
11        if (c + 1 < n) pq.offer(new int[]{matrix[r][c+1], r, c+1});
12    }
13    return pq.peek()[0];
14 }
```

### Step Deduction – Multi-Way Frontier

Treat each matrix row as a sorted stream. The heap maintains the active frontier of length  $\leq N$ . Popping  $K - 1$  times advances exactly to the  $K$ -th smallest.



# Variation 10: Minimum Cost to Hire $K$ Workers

```
1 double mincostToHireWorkers(int[] q, int[] w, int k) {
2     int n = q.length;
3     double[][] arr = new double[n][2];
4     for (int i = 0; i < n; i++)
5         arr[i] = new double[] {(double)w[i]/q[i], q[i]};
6     Arrays.sort(arr, (a, b) ->
7         Double.compare(a[0], b[0]));
8     PriorityQueue<Double> maxHeap =
9         new PriorityQueue<>(Collections.reverseOrder());
10    double qSum = 0, minCost = Double.MAX_VALUE;
11    for (double[] worker : arr) {
12        qSum += worker[1];
13        maxHeap.offer(worker[1]);
14        if (maxHeap.size() > k)
15            qSum -= maxHeap.poll();
16        if (maxHeap.size() == k)
17            minCost = Math.min(minCost, qSum * worker[0]);
18    }
19    return minCost;
20 }
```

## Step Deduction – Ratio Sorting + Max Quality Eviction

Sort workers by wage/quality ratio. For each ratio, we greedily pick the  $K$  smallest qualities using a Max-Heap of size  $K$  to minimize total quality sum.

# Master Decision Matrix: Heap Problem Taxonomy

## Taxonomy of 10 Master Heap & Priority Queue Problems

| Problem Variation               | Archetype           | Heap Type & Bound               | Core Heuristic                                           | Time / Space                              |
|---------------------------------|---------------------|---------------------------------|----------------------------------------------------------|-------------------------------------------|
| 1. <b>K-th Largest</b>          | Top-K Bounded       | Min-Heap of size $K$            | Evict min on overflow $\implies$ top retains $K$ largest | $\mathcal{O}(N \log K) / \mathcal{O}(K)$  |
| 2. <b>Top K Frequent</b>        | Top-K Bounded       | Min-Heap of size $K$            | Compare map frequency counts                             | $\mathcal{O}(N \log K) / \mathcal{O}(N)$  |
| 3. <b>K Closest Points</b>      | Top-K Bounded       | Max-Heap of size $K$            | Evict farthest Euclidean distance                        | $\mathcal{O}(N \log K) / \mathcal{O}(K)$  |
| 4. <b>Connect Ropes</b>         | Greedy 2-by-2       | Min-Heap of size $N$            | Merge 2 shortest ropes continuously                      | $\mathcal{O}(N \log N) / \mathcal{O}(N)$  |
| 5. <b>Merge K Lists</b>         | K-Way Merge         | Min-Heap of size $K$            | Pop min node, push its next child                        | $\mathcal{O}(N \log K) / \mathcal{O}(K)$  |
| 6. <b>Data Stream Median</b>    | Dual-Heap           | Max-Heap (50%) + Min-Heap (50%) | Maintain size balance $ L  -  R  \leq 1$                 | $\mathcal{O}(\log N) / \mathcal{O}(N)$    |
| 7. <b>Reorganize String</b>     | Greedy Cooldown     | Max-Heap of size 26             | Pick highest frequency, cooldown by 1 turn               | $\mathcal{O}(N \log 26) / \mathcal{O}(1)$ |
| 8. <b>Task Scheduler</b>        | Greedy Epochs       | Max-Heap of size 26             | Schedule top tasks in $(n + 1)$ epochs                   | $\mathcal{O}(N) / \mathcal{O}(1)$         |
| 9. <b>Matrix K-th Smallest</b>  | Multi-Way Frontier  | Min-Heap of size $\min(N, K)$   | Advance column index in popped row                       | $\mathcal{O}(K \log N) / \mathcal{O}(N)$  |
| 10. <b>Cost to Hire Workers</b> | Ratio Sweep + Top-K | Max-Heap of size $K$            | Fix ratio, minimize quality sum                          | $\mathcal{O}(N \log N) / \mathcal{O}(K)$  |

# Chapter 04 Synthesis: 7 Core Mental Models

## 1. Subtrees

**Decompose:**  $L + R + \text{root}$ .

- Height, Invert, Sum
- $\mathcal{O}(N)$  Time

## 2. Traversals

**Linearize:** BFS / DFS.

- levelSize barrier
- Pre / In / Post

## 3. Catalan Count

**Partition:**  $C_N = \sum C_i C_{N-1-i}$ .

- Unique BSTs
- Dyck Paths

## 4. LCA Split

**Bubble Up:** Postorder.

- Path dist  $d_1 + d_2$
- Cousins & Turns

## 5. Paths & Geodesics

**Split vs Return:** Apex turns.

- Diameter ( $L + R$ )
- Node & Leaf Path Sums

## 6. Binary Search Trees

**Sorted Range:**  $(\min, \max)$ .

- Search / Insert / Delete
- Monotonic Inorder stream

## 7. Heaps & Priority Queues

**Top-K Bounded:** Array heap.

- Min-Heap for Top K Max
- K-Way merge & Dual heaps

**Invariant**

Mastering these 7 hierarchical invariants gives you complete, world-class mastery across every tree interview challenge.

# Chapter 04 Mastery: The 7 Foundational Tree Invariants

## Summary of Master Invariants in Binary Trees & Heaps

1. **Subtree Composition:** Global tree properties decompose into left/right subtree answers ( $L + R + \text{root}$ ) in  $\mathcal{O}(N)$  time.
2. **The Level-Switch Invariant:** `int levelSize = queue.size()` creates an airtight barrier between horizontal BFS levels.
3. **Euler Tour Flank Principle:** Preorder processes on arrival, Inorder between subtrees, and Postorder on departure.
4. **Catalan Convolution Invariant:** Fixing 1 root node partitions  $N - 1$  remaining nodes into  $C_i \times C_{N-1-i}$  isomorphic configurations.
5. **The Split vs. Return Invariant:** Tree geodesics update global optimums by splitting at apex nodes ( $L + R$ ) while returning linearly ( $\max(L, R) + 1$ ).
6. **The BST Global Range Invariant:** Every node in a BST is strictly bounded by an ancestor interval ( $\min, \max$ ), with Inorder traversal yielding a strictly monotonic sequence.
7. **The Bounded Heap Invariant:** A Min-Heap of capacity  $K$  continuously evicts smaller elements, retaining the Top  $K$  largest elements in  $\mathcal{O}(N \log K)$  time and  $\mathcal{O}(K)$  memory.